

SERVIÇO PÚBLICO FEDERAL
DEPARTAMENTO DE CIÊNCIA E TECNOLOGIA AEROESPACIAL
INSTITUTO TECNOLÓGICO DE AERONÁUTICA



Portal Institucional do ITA

Manual Técnico

ITA - Instituto Tecnológico de Aeronáutica

IEC - Divisão de Ciência da Computação

Carla Cursino

3 de outubro de 2024

Capítulo 1

Arquitetura

1.1 Arquitetura

1.1.1 Decisões Arquiteturais

Todas as necessidades para implementação do Portal e o Painel Administrativo passaram por um processo decisório onde aplicou-se o recurso de registrar um cartão chamado de Registro de Decisão Arquitetural[**kopp2018markdown**].

O formato adotado foi o seguinte:

```
Solução 01:
no <contexto da user story>
face <o problema>
decidimos <a solução>
e negligenciamos <outras possíveis soluções>
para atingir <fatores prós: qualidades e conseqüências desejadas>
aceitando <fatores contras: qualidades e conseqüências indesejadas>
porque <razões, justificativas e raciocínio>
```

Código Fonte 1.1: Exemplo de Registro de Decisão Arquitetural

Armazenamento de Dados

```
Solução 01:
no <armazenamento de conteúdo dinâmico do Portal>
face <a questão de se utilizar dados não estruturados>
decidimos <utilizar um banco de dados NoSQL de código livre>
e negligenciamos <bancos de dados relacionais ou híbridos>
para atingir <fatores prós:
    1) facilidade de permitir aos operadores do Portal a inclusão, exclusão e modificação da estrutura de dados,
    2) facilidade em criar interfaces ou utilizar interfaces prontas para permitir a inclusão, exclusão e modificação de registros no banco de dados,
    3) não ter de utilizar estruturas complexas de metadados, ou até mesmo criar uma estrutura de metadados para atingir os objetivos acima delineados
>
aceitando <fatores contras:
    1) o fato de que bancos de dados NoSQL são não determinísticos (negligenciável se levarmos em consideração que a base de dados não irá chegar a ter centenas de milhares de registros)
    2) bancos de dados NoSQL não são transacionais (negligenciável pois o Portal, por um lado, será uma aplicação de muita leitura e pouca gravação, e por outro lado a possibilidade de haver disputa de dois usuários tentando atualizar o mesmo registro é negligenciável)
>
porque <
    1) bancos de dados NoSQL são infinitamente mais simples de operar do que os bancos relacionais equivalentes
    2) bancos de dados NoSQL são infinitamente mais flexíveis e tolerantes a modificações na estrutura da dados (na verdade, bancos de dados NoSQL não possuem estrutura de dados pré-definidas)
    3) a maioria das soluções NoSQL se encaixam muito bem em arquiteturas de aplicações para internet como é a do Portal
>
```

Código Fonte 1.2: RDA Banco de Dados

Com base nos critérios acima escolhemos o gerenciador de banco de dados *NoSQL* **MongoDB** que com base em nossa experiência anterior atende a todos listados.

Gerenciamento de Conteúdo

```
Solução 02:
na <seleção de um framework para o gerenciamento de conteúdo do Portal>
face <a questão de se utilizar aplicativos de gerenciamento de conteúdo (CMS - Content Management Systems) ou frameworks voltados à construção de gerenciadores de conteúdos>
decidimos <pela adoção de um framework voltado a construção de um gerenciador de conteúdo>
para atingir <fatores prós:
    1) código livre e escrito em linguagem de programação entre as dez melhores classificadas no índice TIOBE,
    2) simplicidade de uso pelo usuário final,
    3) simplicidade de manutenção pelos técnicos responsáveis por manter o site operacional, e
    4) flexibilidade no atendimento específico e pontual de necessidades da instituição
>
aceitando <fatores contras:
    1) a necessidade de que possíveis evoluções do site demandariam conhecimentos de programação e
    2) a possibilidade de não se encontrar soluções prontas para determinadas facilidades
>
porque <
    1) soluções como Drupal ou Wordpress prometem mas não cumprem a possibilidade de construir e manter um site de conteúdo sem a necessidade de programação,
    2) muitas das soluções prontas não são compatíveis entre si e muitos fornecedores adicionam recursos adicionais que para serem utilizados devem ser pagos, e
    3) apesar de pagos a qualidade é algumas vezes duvidosa, a configuração dos recursos adicionais é complexa e em nossa experiência trabalhosa e cheia de percalços
>
```

Código Fonte 1.3: RDA Framework CMS

Com base nos critérios acima foram considerados dois *frameworks*:

1. Wagtail - Linguagem **Python** - completude e complexidade alta
2. KapstoneJS - Linguagem **JavaScript** - completude baixa e complexidade baixa

Escolhemos o *framework* **KeystoneJS**, que algum tempo depois de iniciado o projeto evoluiu para uma nova versão incompatível com a que estávamos utilizando e sem o provisionamento de um plano de migração. Como um dos critérios foi justamente o de que o *framework* escolhido deveria ser escrito numa linguagem reconhecida pelo mercado e de código aberto e com o investimento em tempo e aprendizado já feito na ferramenta, decimos pela *derivação* do projeto original e assim nasceu o **CapstoneJS**.

Seguindo a filosofia proposta, todas as funcionalides pedidas pelos usuários da instituição foram implementadas utilizando recursos do projeto original, e para atendê-las foram criados sub-projetos, alguns derivados de projetos originais voltados ao **KeystoneJS**, outros adaptados para funcionarem tanto neste quanto em nossa derivação, e que listamos:

1. CapstoneJS - Gerenciador de Conteúdo
2. capstone-file-manager - Gerenciador de arquivos dos usuários
3. capstone-tinymce - Integração do editor de textos TinyMCE ao CapstoneJS
4. capstone-intl - Suporte multi-linguagem
5. capstone-accesscontrol - Controle de acesso à modificação de conteúdo por meio de regras
6. capstone-auth - Authenticação com serviço LDAP ou Active Directory

Capítulo 2

Guia do Operador

2.1 Guia do Operador

2.1.1 Sistemas Operacionais

O Portal Institucional do ITA foi desenvolvido sub um sistema operacional compatível com **Unix**, porém não deve apresentar desafio algum para quem utilizar outro tipo de sistema operacional, como o **Windows** por exemplo.

Este guia dedica-se exclusivamente à operação do portal e nenhuma referência à dependências de sistema operacional é feita.

2.1.2 Inicialização do serviço do portal e Painel Administrativo

O Portal Institucional do ITA foi desenvolvido em **JavaScript** e roda sob **NodeJS**. Para maiores detalhes pode ser consultado o Guia do Desenvolvedor.

A inicialização é feita pelo "*Script*" chamado $server.js$ que se encontra na raiz do diretório do portal.

Por conta das primeira definições feitas neste "*Script*", estas definições não são customizáveis.

- Pasta $config$ - esta pasta contém as configurações do portal
- Arquivo $.env$ - este arquivo contém a definição da configuração geral desejada[i]

O "*Script*" de inicialização como serviço de sistema operacional é o padrão adotado no Linux, tanto **Debian** quanto **RedHat**.

[i] O "*Script*" de execução define uma variável que contém o arquivo de configuração geral ativado durante a execução como serviço pelo sistema operacional.

```
1   #!/bin/sh
2
3   APP_NAME="portal"
4   USER="cursino"
5   GROUP="$USER"
6   NODE_ENV="test"
7   PORT="3001"
8   APP_DIR="/home/cursino/portal/site01"
9   NODE_APP="server.js"
10  KWARGS=""
11  CONFIG_DIR="$APP_DIR"
12  PID_DIR="$APP_DIR/pid"
13  PID_FILE="$PID_DIR/$APP_NAME.pid"
14  LOG_DIR="$APP_DIR/log"
15  LOG_FILE="$LOG_DIR/$APP_NAME.log"
16  NODE_EXEC=$(which node)
17
18  ###############
19
20  # REDHAT chkconfig header
21
22  # chkconfig: - 58 74
23  # description: node-app is the script for starting a node app on boot.
24  ### BEGIN INIT INFO
25  # Provides: node
26  # Required-Start:    $network $remote_fs $local_fs
27  # Required-Stop:     $network $remote_fs $local_fs
28  # Default-Start:     2 3 4 5
29  # Default-Stop:      0 1 6
30  # Short-Description: start and stop node
31  # Description: Node process for app
32  ### END INIT INFO
33
34  ###############
35
```

Código Fonte 2.1: "*Script*" de Inicialização

Entre as linhas 1 e 35 temos a inicialização das variáveis de ambiente, com destaque a linha 6 onde definimos o arquivo de configuração da pasta $config$ que será carregado durante a inicialização.

```
57  start_it() {
58      mkdir -p "$PID_DIR"
59      chown $USER:$GROUP "$PID_DIR"
60      mkdir -p "$LOG_DIR"
61      chown $USER:$GROUP "$LOG_DIR"
62
63      echo "Starting $APP_NAME ..."
64      echo "cd $APP_DIR
65          if [ $? -ne 0 ]; then
66            exit
67          fi
68          set -a
69          PORT=$PORT
70          NODE_ENV=$NODE_ENV
71          NODE_CONFIG_DIR=$CONFIG_DIR
72          $NODE_EXEC \"$APP_DIR\"/$NODE_APP $KWARGS &>> \"$LOG_FILE\" &
73          echo \$! > $PID_FILE" | sudo -i -u $USER
74      echo "$APP_NAME started with pid $(get_pid)"
75  }
```

Código Fonte 2.2: "*Script*" de Inicialização

Da linha 57 até a linha 75 temos a inicialização do serviço **NodeJS** e o "*Script*" da aplicação definido nas linhas 8 e 9 é chamado.

Configuração

A configuração do portal se faz por meio de variáveis de ambiente e arquivos de configuração. O primeiro arquivo de configuração é inicializado pelo "*Script*" $server.js$ que lê o arquivo $.env$ e determina qual o arquivo de configuração seguinte que deve ser lido. Isto permite que várias configurações sejam criadas e escolhidas pelo operador. Por exemplo, existe uma configuração na pasta $config$ chamada $maintenance.js$ que serve para colocar o portal em modo de manutenção (apresenta uma página informando aos usuários que o portal está fora do ar).

```
1  NODE_ENV=validation
```

Código Fonte 2.3: Definição do arquivo de configuração

A configuração utilizada no desenvolvimento chama-se $validation.js$, enquanto que a utilizada no ambiente de testes do ITA chama-se $test.js$. Uma nova configuração chamada $production.js$ existe para o ambiente de produção.

> *É responsabilidade do operador, em conjunto com o desenvolvedor, manter estes arquivos de configuração. Recomenda-se que o arquivo* $production.js$ *seja protegido pelo operador e modificado apenas por este.*

```
module.exports = {
    cms: {
        'mongo': 'mongodb://172.17.1.30:27017/portalita',
        'cookie secret': 'MEVlYiAtykSik1hZfEJgqg',

        'frontPageCategories': { categories: { $in: ['600456f37362146fd66e02fe'] } },
    },
    portal: {
        ita: {
            instituicao: {
                informacoesGerais: '/geral',
                visaoFuturo: '/post/visao-e-futuro',
                legislacao: '/post/legislacao',
                historiaValores: {
                    concepcao: '/post/a-concepcao',
                    construcao: '/post/construcao',
                    criacaoCursos: '/post/criacao-dos-cursos',
                    valoresITA: '/post/valores-do-ita',
                },
            },
            localizacao: {
                googleFrameMap: 'https://maps.google.com/maps?q=instituto%20tecnol%C3%B3gico%20de%20aeron%C
                ↪ 3%A1utica&t=&z=13&ie=UTF8&iwloc=&output=embed',
                googleLinkMap: 'https://www.google.com/maps?ll=-23.210871,-45.875305&z=13&t=m&hl=en-US&gl=U
                ↪ S&mapclient=embed&cid=4460331161721657284',
            },
            organizacao: {
                organograma: '/post/organograma',
                reitoria: '/post/reitoria',
                congregacao: '/post/congregacao',
                divisoes: '/post/divisoes',
                proReitorias: {
                    graduacao: '/post/graduacao',
                    posGraduacao: '/post/pro-reitoria-de-pos-graduacao',
                    pesquisaRelacionamentoInstitucional: '/post/pro-reitoria-ipr',
                    administracao: '/post/pro-reitoria-de-administracao',
                },
            },
        },
        contato: {
            webmail: 'https://webmail.ita.br',
            contatos: {
                mail: 'mailto:contato@ita.br',
                comunicacao: 'mailto:comunicacao@ita.br',
                vestibular: 'mailto:vestita@ita.br',
                ramais: 'http://www.portal2.ita.br/contatos',
                telefones: '/post/contatos',
                foneAdministracao: '(12) 3947 5736',
                foneComunicacao: '(12) 3947 6926'
            },
            privacidade: {
                politica: '/post/politica-de-privacidade',
                cookies: '/post/politica-de-cookies',
            },
            termos: {
                uso: '/post/termo-de-uso',
            },
        },
    },
}
```

Código Fonte 2.4: Arquivo de configuração do ambiente de desenvolvimento

*Uma atenção especial deve ser dada ao arquivo de configuração $default.js$. Toda e qualquer configuração que exister em $default.js$ será **sobreposta** por $validation.js$ (ou $test.js$ ou $production.js$) se a mesma configuração existir neste último. Porém, caso a configuração não seja definida para nenhum destes arquivos, então a configuração em $default.js$ irá prevalecer.*

Na Listagem 2.4 mostramos a configuração do ambiente de desenvolvimento. As configurações são válidas para um ambiente de produção, importanto apenas os valores informados, portanto usaremos

esta como exemplo.

Uma configuração importante é a do banco de dados **MongoDB** a ser utilizado (esta informação é importante para ser utilizada nas rotinas de "*Backup*" que abordaremos mais a seguir). Na linha 3 temos a chamada "*connection string*" do **MongoDB** padrão. Em seguida temos uma chave criptográfica aleatória que serve para assinar os "*cookies*" de sessão do portal[ii]

Em seguida, na linha 6 é feita uma configuração básica de controle de acessos - a definição de quais categorias irão compor a página inicial do portal. Os códigos das categorias desejadas, deve ser o $_Id$ (ver documentação do **MongoDB** sobre atributos internos do documento) de cada uma (é possível usar uma *Array* de *Strings JavaScript* para várias valores neste parâmetro).

O **CapstoneJS** irá usar este parâmetro para determinar quais publicações poderão aparecer na página principal, e a princípio, o controle de acessos deve ser configurado para determinar quem poderá incluir publicações nestas categorias (ver manual correspondente).

Em seguida, entre as linhas 9 e 37 são definidas publicações utilizadas para diversos ítens do menu do portal. Em particular, entre as linhas 38 e 56 definimos os valores para o painel de contato do portal.

Este foi apenas um exemplo, e muitos destes valores estão também definidos no arquivo $default.js$ e servem apenas para ilustrar o fato de que os valores em $default.js$ serão sobrepostos por estes no ambiente do desenvolvedor, e poderão sê-lo também no ambiente de produção. Uma configuração mínima pode ser a da Listagem 2.5 por exemplo.

```
1  module.exports = {
2      cms: {
3          'mongo': 'mongodb+srv://portalAdmin:<password>@cluster0.xvymj.mongodb.net/<dbname>?retryWrites=true
           ↪  &w=majority',
4          'headless': true,
5          'cookie secret': 'MEVlYiAtykSik1hZfEJgqg',
6      },
7  }
```

Código Fonte 2.5: Arquivo de configuração do banco de dados

Vamos agora entrar em maiores detalhes nos parâmetros mais importantes e vamos listar o arquivo $default.js$.

```
1  require('app-module-path').addPath(__dirname + '/helpers')
2
3  const engine = require('express-handlebars')
4  const handlebarsHelpers = require('handlebars-helpers')()
5  const templateHelpers = require('template-helpers')()
6  const helperMoment = require('helper-moment')
7
8  const log = require('logger')
9
10 const UPLOADS = './assets/static/core/uploads'
```

Código Fonte 2.6: Definições para o mecanismo de configuração do portal

Entre as linhas 3 e 8 definimos os componentes utilizados pelo mecanismo de configuração do portal.

[ii] Uma nota importante a ser feita aqui é que em momento algum o portal guarda informações pessoais de seja quem for que estiver navegando por suas páginas e este "*cookie*" é criado para ser usado pelo Painel Administrativo, e portanto **não é necessário pedir consentimento do usuário para este cookie** pois nada será compartilhado com terceiros. Observe que isto deixa de ser verdade se eventualmente o portal for modificado para inclusão de "*Scripts*" de terceiros como *trackers* do Google e conexões com redes sociais.

Na linha 10 definimos em que diretório desejamos “despejar” os arquivos que os editores colocarem à disposição dos usuários para baixarem em seus computadores (esta informação é importante para ser utilizada nas rotinas de “*Backup*” que abordaremos mais a seguir).

```
12  module.exports = {
13      storage: {
14          uploads: UPLOADS,
15      },
16      cms: {
17          'name': 'Portal ITA',
18          'brand': 'ITA',
19          'favicon': 'assets/static/favicon.ico',
20          'static': 'assets',
21          'compress': true,
22
23          'auto update': true,
24          'user model': 'User',
25
26          'port': 3001,
27          'session': true,
28          'session store': 'mongo',
29          'auth': true,
30          'admin path': 'admin',
31          'signin logo': '/static/images/logo-ita.png',
32
33          // 'ldap server': {
34          //     url: 'ldap://172.17.1.30:389',
35          // },
36          'ldap auth user': 'cn=admin,dc=ita,dc=br',
37          'ldap auth password': 'p4ssw0rd',
38          'ldap query base': 'dc=portal,dc=ita,dc=br',
39          'ldap query user': {
40              filter: '(objectClass=account)',
41              scope: 'sub',
42              attrs: '*',
43          },
44          'ldap query role': {
45              filter: '(objectClass=groupOfNames)',
46              scope: 'sub',
47              attrs: '*',
48          },
49          'ldap default role': 'cn=admin,dc=portal,dc=ita,dc=br',
50
51          'applog': log.applog,
52          'seclog': log.seclog,
53
54          'logger': 'combined',
55          'logger options': {
56              stream: log.reqlog.stream
57          },
58
59          'kfm public url': '/static/core/uploads/',
60          'kfm uploaded files storage': UPLOADS,
61
62          'supported languages': ['pt', 'en'],
63          'language': 'pt',
64
65          'cors allow methods': false,
66          'cors allow headers': false,
67
68          'views': 'views',
69
70          'custom engine': engine({
71              helpers: [
72                  handlebarsHelpers,
73                  templateHelpers,
74                  helperMoment,
75              ],
76              layoutsDir: 'views/layouts',
77              partialsDir: 'views/partials',
78              defaultLayout: 'default.handlebars',
79          }),
80          'view engine': 'handlebars',
81
82          'wysiwyg additional options': { extended_valid_elements : 'pre[*]' },
83      },
84      models: './models',
85      routes: './routes',
86  }
```

Código Fonte 2.7: Configuração geral do portal

Logo na linha 14 utilizamos a definição que fizemos na linha 10.

Em seguida, fazemos definições gerais para o $CapstoneJS$. Informações para a maioria destes parâmetros pode ser encontrada no site do projeto de referência ($KeystoneJS - versão 4$), mas destacaremos os mais importantes:

20. Conteúdo estático - $static$ aponta para a pasta onde estará o conteúdo estático do portal, como arquivos **JavaScript** e folhas de estilo **CSS** que serão servidos ao cliente por meio do "*Browser*"
23. Atualização automática - esta opção irá ler um "*Script*" em **JavaScript** na pasta denominada $update$ contendo dados no formato **JSON** para atualizar o banco de dados (veja exemplo na Listagem 2.8)
24. Registros de usuários - definine o nome da estrutura de dados que irá conter o registro de usuários do painel de controle do portal (ver Guia do Desenvolvedor)
26. Porta - define a porta de comunicação para o servidor de aplicações **NodeJS**
27.
28. Ativa o armazenamento da sessão num serviço externo ao próprio portal[iii] A sessão utilizada pelo portal não armazena dados pessoais do usuário e não é utilizada de modo algum para rastreamento do usuário pela internet.)
29. Armazenamento de sessões - define o serviço de dados que irá conter registro das sessões do Painel Administrativo
30. *URI (Uniform Resource Identifier, ou indentificador de recurso num servidor na internet, é uma referência para o recurso, podendo ser um documento, imagem ou mídias tais como som ou vídeo)* do Painel Administrativo - caminho que a partir do endereço do portal podemos acessar o Painel Administrativo
31. Logotipo da organização

```
1   exports.create = {
2     User: [
3       {
4         displayName: 'admin',
5         email: 'admin@local.io',
6         password: 'p4ssw0rd',
7         isAdmin: true,
8       },
9     ],
10  }
```

Código Fonte 2.8: Exemplo de inclusão do primeiro usuário no banco de dados do portal

Uma atenção especial deve ser dada à configuração da autenticação com um servidor *LDAP (Lightweight Directory Access Protocol)*.

Caso não exista um servidor *LDAP* disponível, as linhas de 33 até 35 devem ser comentadas (como no exemplo na Listagem 2.7) e o mecanismo de controle de acesso irá automaticamente usar a definição de acesso do parâmetro $ldapdefaultrole$. A configuração em $ldapdefaultrole$ deve ser definida no arquivo $rules.js$ como todas as demais (ver Controle de Acesso).

> *No exemplo foi usada* $'cn = admin, dc = portal, dc = ita, dc = br'$ *que em rules está definida como um usuário de acesso total ao Painel, o que pode ser útil para fazer ajustes emergenciais de curto prazo, mas que não é recomendado fazer para o Portal em produção.*

[iii] (

Em seguida, entre as linha 51 e 57 temos a definição dos arquivos que irão conter os registros de eventos do portal que serão repassados ao $CapstoneJS$.

nas linhas 59 e 60 temos mais definições para o gerenciador de arquivos. Na linha 62 temos as linguagens disponíveis para o módulo de traduções e na linha 63 a linguagem a ser usada como padrão pelo portal.

Todas as demais linhas são definições do **NodeJS** e **ExpressJS** (ver Guia do Desenvolvedor).

2.1.3 Backup

De modo geral, o Portal Institucional é mantido pelos desenvolvedores num repositório de fontes que é versionado e portanto não deve haver necessidade de se manter cópias de segurança do próprio conteúdo estático.

Porém o conteúdo dinâmico (publicações, imagens e arquivos dos usuários) devem ser copiados e mantidos em segurança.

Outro elemento do qual também pode ser desejável manter cópias para referências futuras são os registros de eventos (pasta log dentro do portal).

Assim sendo, as definições de conexão com o servidor **MongoDB**, local dos arquivos disponibilizados pelos editores (definida na linha 10 do arquivo de configuração $default.js$) e a pasta log do portal devem ser salvas pelas rotinas diárias de "*Backup*".

2.1.4 Registros de eventos

Tanto o Portal Institucioanl quanto o Painel Administrativo possuem quatro registros de eventos:

- "*Log*" da aplicação - registra eventos gerais que ocorrem com a aplicação
- "*Log*" de segurança - registra os eventos de segurança
- "*Log*" de requisições - registra todas as requisições feitas ao portal e ao painel
- "*Log*" de erros - registra ocorrências de erros (que também pode ser registrados nos "*logs*" de aplicação e segurança para facilitar o rastreamento)

A configuração dos registros de eventos é feita programaticamente no "*Script*" $helpers/logger.js$, e lá estão definidos os seguintes parâmetros para todos os arquivos:

- Nível de registros ($level$): $'info'$
- Nome e local do arquivo ($filename$): pasta log, arquivo:
  - Tipo do log:
    * app - Aplicação
    * sec - Segurança
    * req - Requisições
    * err - Erros
  - Data e hora do registro no formato:

    * Ano com quatro dígitos
    * Mês com dois dígitos
    * Dia com dois dígitos
    * Hora com dois dígitos
  - Tipo de arquivo **JSON**

- Tamanho máximo: 20MB
- Tempo máximo: 14 dias
- Arquivo compactado ($zippedArchive$) - quando o arquivo atinge o tamanho ou o tempo máximo (o primeiro que ocorrer) será compactado
- Tratamento de exceções - automaticamente grava um registro quando ocorrem exceções **JavaScript**

O **CapstoneJS** utiliza apenas os registros de eventos de aplicação e de segurança. Já o portal e o painel utilizam todos os registros[iv]

[iv] Ocorrências de erros em qualquer componente de terceiros utilizado pelo portal ou pelo painel, mesmo fazendo parte do conjunto do **CapstoneJS**, são apresentado no console e o "*script*" de inicialização do serviço deve tomar o cuidado de salvar este console apropriadamente de acordo com o que for recomendado pelo sistema operacional em uso.

Capítulo 3

Guia do Desenvolvedor

3.1 Guia do Desenvolvedor

Neste guia serão apresentados procedimentos essenciais para o desenvolvedor que desejar customizar o Portal Institucional do ITA.

Para tal é necessário conhecimentos de:

- **JavaScript** - https://developer.mozilla.org/en-US/docs/Web/javascript
- **NodeJS** - https://nodejs.org/en/
- **HTML** - https://www.w3.org/html/
- **CSS** - https://www.w3.org/Style/CSS/Overview.en.html
- **MongoDB** - https://www.w3.org/Style/CSS/Overview.en.html

Além destes elementos essenciais, bibliotecas JavaScript são utilizadas para facilitar o uso destas tecnologias, e entre estas destacamos as seguintes ferramentas que também devem ser dominadas pelo desenvolvedor:

- **ExpressJS** - Interface de programação entre a plataforma NodeJS e cliente no browser - http://expressjs.com
- **HandlebarsJS** - Templates de páginas HTML - https://handlebarsjs.com
- **Mongoose** - Interface de programação entre o banco de dados de documentos Mongo e aplicações NodeJS - https://mongoosejs.com
- **Winston** - Registro de eventos ("*Logs*") - https://github.com/winstonjs/winston

De modo algum este guia pretende servir de instrução ou sequer introdução a qualquer uma das tecnologias listadas e toda e qualquer informação necessária para uso e configuração das mesmas deve ser obtido diretamente nas fontes apontadas acima ou em fontes adicionais que podem ser encontradas na internet

***Atenção**: Listagens de trechos de código, "scripts" e páginas HTML apresentados neste manual podem não corresponder à versão final do portal.*

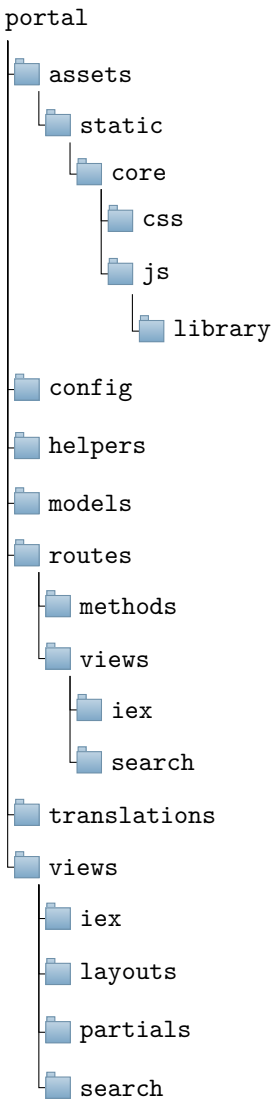
Algumas das providencias tomadas para documentar o portal, foram por exemplo, reformatar o código para melhor apresentação neste manual, portanto diferenças básicas são esperadas.

De qualquer modo, a versão utilizada para confecção deste manual é uma versão totalmente funcional, livre de falhas significativas e de acordo com os desejos dos tomadores de decisão que nos orientaram durante a construção do portal,

Qualquer diferença notada pode e deve ser considerada irrelevante a partir deste momento.

3.1.1 Localizando-se

Estrutura de diretórios



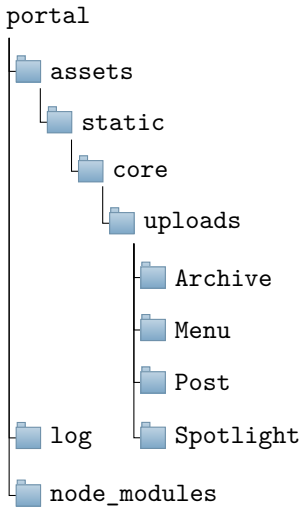
Estas pastas contém programas do portal e artefatos da camada de apresentação do portal.

- "*assets / static* " - o conteúdo "estático" do portal, ou seja, que não é formatado por código do portal
- "*config*" - arquivos de configuração

- "*helpers*" - rotinas de suporte comuns à diversas funções do portal
- "*models*" - definições da estrutura de dados do portal (o conteúdo dinâmico produzido pelo portal)
- "*routes*" - código do portal que processa as rotas de requisições (*URL (Uniform Resource Locator, ou endereço internet, é uma referência completa para um recurso localizado num servidor na internet, podendo ser um documento, imagem ou mídias tais como som ou vídeo)*) do navegador
- "*translations*" - tradução dos termos comuns do portal (a tradução de páginas é feita à parte por meio da estrutura de dados)
- "*views*" - definições da camada de apresentação (contém a forma e direciona o conteúdo apresentado pelo portal)

Estas são as principais pastas com que o desenvolvedor deve se ocupar para customizar o Portal Institucional. Maiores detalhes serão apresentados nas seções à seguir.

Além destas, outras pastas de trabalho são criadas para conteúdo e bibliotecas do Portal.



- "*assets / core / uploads*" - contém arquivos que foram inseridos no conteúdo do portal por editores
- "*log*" - arquivos com registros de ocorrências do portal

Estes arquivos podem ser configurados e residir em outra estrutura de arquivos do servidor.

3.1.2 Configuração

O Portal Institucional possui diversos arquivos de configuração e parte da customização é feita por meio de ajustes nestes arquivos.



Além destes arquivos, o "*Script*" de inicialização do sistema operacional carrega definições particulares do ambiente de execução (ver Guia do Operador).

- $cookies.js$ - habilita a opção de apresentar uma mensagem solicitando permissão de uso de "*Cookies*" pelo Portal
- $default.js$ - configurações comuns e gerais
- $editor.js$ - configuração do editor de texto utilizado para criar e atualizar publicações
- $headers.js$ - configuração dos cabeçalhos *HTML (Linguagem de formatação de documentos, do inglês "Hypertext Markup Language é, do inglês" (Linguagem de Marcação Hipertexto))*
- $rules.js$ - configurações de regras de acesso

3.1.3 Customizando o Portal

O Portal Institucional é amplamente customizável por um desenvolvedor com conhecimentos de HTML e CSS e algum conhecimento básico de JavaScript.

Um desenvolvedor com conhecimento mais profundo de JavaScript e a plataforma NodeJS poderá transformar totalmente o portal.

Antes de mais nada, para construção do portal, foram utilizados algumas ferramentas básicas, todas elas incorporadas e customizadas, ou seja, o Portal Institucional é o resultado de ampla e profunda customização de outros projetos de código livre na internet.

A principal ferramenta utilizada é o *CMS (Content Management System, um sistema para gestão de conteúdo para portais na internet)* Keystone. Nossa customização chama-se "**CapstoneJS**" e foi baseada na versão 4 do projeto "**KeystoneJS**" original. Tanto que a documentação original daquele projeto pode ser utilizada como referência para a nossa versão (veja em https://v4.keystonejs.com/documentation/)[i].

Esta implementação, apesar de permitir a gestão de conteúdo dinâmico para um Portal, não pode ser classificada como um *CMS* completamente funcional pois isto vai muito além do escopo de um Portal de conteúdo para uma instituição de ensino superior, faltando neste alguns dos recursos encontrados em produtos do mercado tais como:

- Taxonomia, ou a criação hierárquica de classes de informações e a classificação de publicações dentro destas classes e a hierarquização de conteúdo

[i] A principal razão para termos realizado o que se chama "*fork*" deste projeto é que a versão 4 foi descontinuada e a versão 5 é completamente incompatível, e como necessitamos implementar melhorias e correções que os desenvolvedores originais não mais estão disponíveis para fazê-las

- Indexação completa e compreensiva por autores, data, taxonomia, etc.
- Gerenciamento de formatos de apresentação
- Controles de revisão de documentos
- Gerenciamento de sistema de arquivos, armazenamento de documentos, conversão de formatos de documentos, importação de documentos
- Gerenciamento de "*templates*" de apresentação de conteúdo, customização de "*templates*"
- Inclusão de comentários de leitores e de grupos de discussão
- Controle de versionamento
- Gerenciamento muli-idiomas customizável

Porém este Portal implementa de modo limitado algumas destas funcionalidades:

- Inclusão de conteúdo dinâmico a ser publicado pelo Portal com um editor *HTML WYSIWYG (Em computação WYSIWYG é, do inglês, "What You See Is What You Get" (O que você vê é o que você obterá))* simples
- Tradução de conteúdo por meio de seleção do idioma desejado no Painel (limitada aos idiomas disponibilizados via configuração - ver Guias de Operação e Desenvolvimento)
- Configuração de regras de acesso e perfis de usuário
- Importação de arquivos para serem baixados pelos leitores
- Importação de imagens para serem apresentadas no conteúdo estático

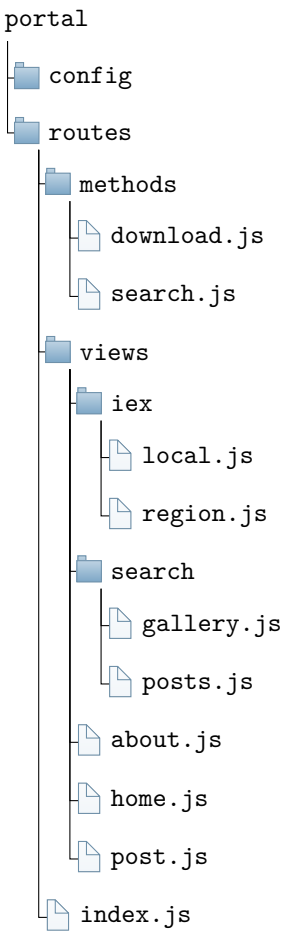
Além do "**CapstoneJS**" implementamos também:

- "*capstone-accesscontrol*" - baseado no projeto "*onury / accesscontrol*" faz controle de acesso para edição de conteúdo do portal integrado ao "*CapstoneJS*"
- "*capstone-file-manager*" - faz a mesma função do componente "*Sidenis / KeystoneJS file manager*" que permite aos editores inserir arquivos para serem baixados pelos usuários que navegam no portal
- "*capstone-intl*" - versão customizada para o "*CapstoneJS*" do componente "*alexsk / mongoose-intl*" que permite inserir conteúdo multi-idioma em estruturas de dados do MongoDB

São principalmente estas mudanças e implementações adicionais que explicaremos aqui, e para maiores informações sobre como usar estes componentes, basta acessar os projetos originais e buscar na documentação.

A construção de uma rota

Rotas são *URIs (Uniform Resource Identifier, ou indentificador de recurso num servidor na internet, é uma referência para o recurso, podendo ser um documento, imagem ou mídias tais como som ou vídeos)* que apresentam o conteúdo do portal de modo específico.



As rotas são "configuradas" no arquivo $config/default.js$ conforme podemos ver na linha 85 da Listagem 3.1.

```
83        },
84        models: './models',
85        routes: './routes',
86    }
```

Código Fonte 3.1: Configuração das rotas

As rotas são "definidas" no arquivo $routes/index.js$ (veja Listagem 3.2).

```
105
106     app.get('/:lang?/search', [capstone.middleware.api, capstone.middleware.cors], routes.methods.search)
107     app.get('/:lang?', [capstone.middleware.api, capstone.middleware.cors], routes.views.home)
108     app.get('/:lang?/gallery/:gallery', [capstone.middleware.api, capstone.middleware.cors],
        ↪  routes.views.search.gallery)
109     app.get('/:lang?/geral', [capstone.middleware.api, capstone.middleware.cors], routes.views.about)
110     app.get('/:lang?/iex/:region', [capstone.middleware.api, capstone.middleware.cors],
        ↪  routes.views.iex.region)
111     app.get('/:lang?/iex/local/:local', [capstone.middleware.api, capstone.middleware.cors],
        ↪  routes.views.iex.local)
112     app.get('/:lang?/post/download/:archive', [capstone.middleware.api, capstone.middleware.cors],
        ↪  routes.methods.download)
113     app.get('/:lang?/post/:post', [capstone.middleware.api, capstone.middleware.cors], routes.views.post)
114     app.get('/:lang?/posts/category/:category', [capstone.middleware.api, capstone.middleware.cors],
        ↪  routes.views.search.posts)
115
```

Código Fonte 3.2: Definição das rotas

A definição da rota segue o método padrão adotado pelo **ExpressJS**, onde o padrão da requisição da *URI* é informado, seguido do "*Middleware*" padrão de proteção do **Capstone** (aka **Keysonte**), seguido do caminho para o programa em **JavaScript** com a implementação da rota.

Aas rotas fixas listadas na Listagem *3.2* se referem à:

116. $search$ - pesquisa de publicações
117. $gallegy$ - listagem de publicações com disposição em quadros com várias linhas e colunas
118. $geral$ - a página de apresentação da instituição
119. iex - lista de regiões dos acordos internacionais da **IPR**
120. $ies/local$ - lista local de acordos da **IPR**
121. $post$ - mostra uma publicação
122. $posts/category$ - lista todos as publicações de uma determinada categoria

Observe que a notação $routes.methods.search$ equivale ao caminho para a pasta $routes/methods/search$

Por exemplo, a rota / : $lang?/geral$ aponta para $routes.views.about$. Nas listagens 3.3, 3.4 e 3.5 vemos a implementação completa.

```
1   require('app-module-path').addPath(__dirname + '/helpers')
2
3   const partials = require('partials')
4   const capstone = require('capstonejs')
5
6   module.exports = (req, res) => {
7       const view = new capstone.View(req, res)
8
9       res.locals.data = {
10          today: new Date(),
11          menu: [],
12          info: {},
13          grad: {},
14          posgrad: {},
15          pg: [],
16          g: [],
17      }
18
```

Código Fonte 3.3: Implementação de uma rota

Em 3.3 fazemos a inicialização da rota. O componente $app - module - path$ facilita localizar todos os componentes que estão na pasta $helpers$. Em seguida instanciamos dois componentes essenciais para toda rota:

- $partials$ - contém "partes" das páginas do portal que juntas formam a apresentação completa do conteúdo desejado
- $capstone$ - o gerenciador de conteúdo

Em seguida, na linha 7 inicializamos o gerenciador de conteúdo (**CapstoneJS**) e da linha 9 até 17 as variáveis que irão conter o conteúdo apresentado.

```
18
19      view.on('init', (next) => {
20          partials.menu(res.locals.language, (err, result) => {
21              res.locals.data.menu = result
22              next(err)
23          })
24      })
25
26      view.on('init', (next) => {
27          partials.post({ panel: 'info' }, res.locals.language, (err, result) => {
28              res.locals.data.info = result
29              next(err)
30          })
31      })
32
33      view.on('init', (next) => {
34          partials.post({ panel: 'grad' }, res.locals.language, (err, result) => {
35              res.locals.data.grad = result
36              next(err)
37          })
38      })
39
40      view.on('init', (next) => {
41          partials.post({ panel: 'posgrad' }, res.locals.language, (err, result) => {
42              res.locals.data.grad = result
43              next(err)
44          })
45      })
46
47      view.on('init', (next) => {
48          partials.posts({ panel: 'pg' }, res.locals.language, (err, result) => {
49              res.locals.data.pg = result
50              next(err)
51          })
52      })
53
54      view.on('init', (next) => {
55          partials.posts({ panel: 'g' }, res.locals.language, (err, result) => {
56              res.locals.data.geral = result
57              next(err)
58          })
59      })
60
```

Código Fonte 3.4: Implementação de uma rota

Prosseguimos resgatando o conteúdo do portal usando as rotinas em $partials$ da linha 19 até 59. O conteúdo resgatado é atribuído às variáveis definidas nas linhas 9 até 17.

```
60
61      view.render('about')
62  }
```

Código Fonte 3.5: Implementação de uma rota

Finalmente, na linha 61, solicitamos ao **CapstoneJS** para apresentar a página desejada.

Na seção a seguir iremos apresentar como resgatar conteúdo parcial e sem seguida como compor uma página.

Resgate de um conteúdo parcial

Vimos que durante o processamento de uma rota, informações dos dados a serem apresentados pelas páginas do portal como conteúdo dinâmico são resgatados em pequenas porções que chamamos de "parciais".

Estas "parciais" estão todas definidas no programa $helpers/partials.js$. Vamos ver a seguir como definir uma parcial.

```
60
61  exports.post = (query, language, callback) => {
62      capstone.list('Post').model.findOne(query)
63          .sort({ publishedDate: -1 })
64          .limit(1)
65          .exec((err, result) => {
66              if (result)
67                  result.setLanguage(language)
68              callback(err, result)
69          })
70  }
71
```

Código Fonte 3.6: Implementação de uma parcial

A Listagem *3.6* lista o resgate de um "*Post*" (postagem ou publicação), que é o elemento central de um gerenciador de conteúdo. Na linha 61 a partial é definida recebendo os parâmetros $query$, $language$ e $callback$ (ela segue o padrão de rotinas assíncronas do **NodeJS**).

Em $query$ recebemos uma expressão que irá pesquisar no banco de dados **MongoDB** pelo "*Post*" que desejamos. Apenas um único "*Post*" será resgatado por esta rotina (outras parciais resgatam mais de um). Esta expressão obedece os critérios de expressões do **MongoDB** e **Mongoose**.

Em seguida o parâmetro $language$ é o código de linguagem para o qual o resultado deve ser traduzido pelo gerenciador de conteúdo. Finalmente, o parâmetro $callback$ contém uma função em **JavaScript** que irá receber a resposta da consulta quando esta estiver pronta (padrão de chamadas assíncronas do **NodeJS**).

Olhando nas linha 27 da Listagem *3.4* podemos ver uma chamada à esta parcial (trecho que reproduzimos à seguir).

```
25
26      view.on('init', (next) => {
27          partials.post({ panel: 'info' }, res.locals.language, (err, result) => {
28              res.locals.data.info = result
29              next(err)
30          })
31      })
32
```

Código Fonte 3.7: Implementação de uma rota

Na chamada à $partials.post$ passamos os seguintes parâmetros:

- $query$ - o valor $panel :' info'$, ou seja, buscamos um "*Post*" cujo atributo $panel$ contenha $'info'$
- $language$ - o valor $res.locals.language$, esta é uma variável que contém a linguagem atual usada pelo $portal$
- $callback$ - o valor (que na verdade é uma função) $(err, result) => ...$

No caso de $callback$ optamos por implementar uma função diretamente dentro da chamada da função $partials.post$ por meio da notação $(err, result) => ...$ do **JavaScript**.

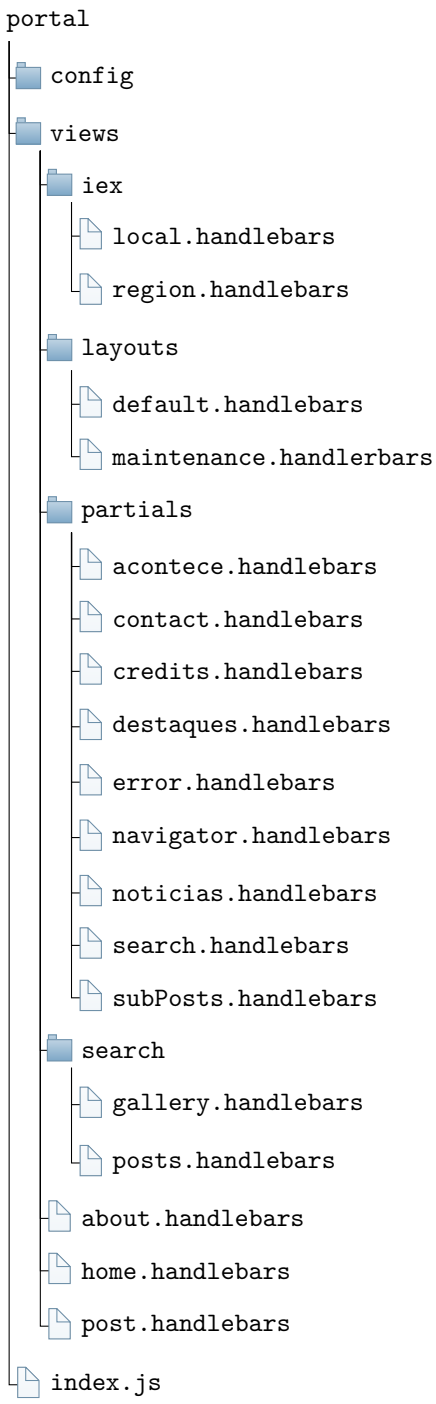
Observe que na linha 22 estamos fazendo a chamada ao "*callback*" do próprio **CapstoneJS** (parâmetro $next$ da função $view.on$ chamada na linha 26) - neste caso chamamos $next$ passando qualquer erro (variável err) que tenha sido retornado pelo nosso próprio "*callback*" da chamada à $partials.post$.

Composição de uma página

Para compor páginas, utilizamos o recurso de criar templates em **HandlebarsJS**, contendo "peças" que ao serem combinadas irão forar uma página completa.

Existem basicamente três tipos de arquivos definodos pelo **HandlebarsJS**:

- "*Partial*" - contém um fragmento de uma página
- "*Layout*" - contém diretivas que irão definir onde cada fragmento será posicionado para compor a página final
- "*Body*" - um tipo de especial de fragmento que será substituído por outro fragmento designado pelo gerenciador de conteúdo ou o "*Middleware*" do **ExpressJS**


portal
config
views
iex
local.handlebars
region.handlebars
layouts
default.handlebars
maintenance.handlerbars
partials
acontece.handlebars
contact.handlebars
credits.handlebars
destaques.handlebars
error.handlebars
navigator.handlebars
noticias.handlebars
search.handlebars
subPosts.handlebars
search
gallery.handlebars
posts.handlebars
about.handlebars
home.handlebars
post.handlebars
index.js

```
67
68          'views': 'views',
69
70          'custom engine': engine({
71              helpers: [
72                  handlebarsHelpers,
73                  templateHelpers,
74                  helperMoment,
75              ],
76              layoutsDir: 'views/layouts',
77              partialsDir: 'views/partials',
78              defaultLayout: 'default.handlebars',
79          }),
80          'view engine': 'handlebars',
81
```

Código Fonte 3.8: Configuração da camada de apresentação

A localização destas "peças" é definida na linha 68, e nas linhas 76, 77 e 78 definimos a localização das peças restantes para a composição da página, assim como o "*layout*" que desejamos usar.

Iremos descrever nas seções à seguir cada um destes elementos.

Layouts de página

Os "*layouts*" de página estão colocados na pasta $views/layouts$ [ii].

[ii] Em geral somente precisamos de um "*layout*" para um portal na internet, mas neste caso temos uma segunda opção que é usada quando o portal está em manutenção e apresentamos uma página de aviso

```
1   <!DOCTYPE html>
2   {{#if language}}
3   <html lang="{{language}}">
4   {{else}}
5   <html lang="pt">
6   {{/if}}
7   {{> head }}
8
9   <body class="index">
10
11          {{> sideOptions }}
12
13          <header id="header">
14
15                  {{> barraBrasil }}
16
17                  {{> nav }}
18
19          </header>
20
21          {{{body}}}
22
23          <footer id="footer">
24
25                  {{> bottom }}
26
27                  {{> footerBrasil }}
28
29                  {{> credits }}
30
31          </footer>
32
33          <a class="btn btn-icon-top" id="toTop" href="#"></a>
34
35          {{> scripts }}
36
37  </body>
38
39  </html>
```

Código Fonte 3.9: Layout do Portal Institucional

Como podemos ver, o "*layout*" é basicamente uma página **HTML** muito simples contendo chaves especiais que irão, por meio do **HandlebarsJS**, resgatar trechos contendo outros fragmentos de página que irão no final produzir a página completa. Podemos ver estas chaves nas linhas 7, 11, 15, 17, 25, 27, 29 e 35.

Na linha 21 temos uma chave especial que designa um ponto chamado *body* que será o ponto de inserção de conteúdo e onde um fragmento designado pelo gestor de conteúdo ou pelo **ExpressJS**.

Parciais da página

As parciais de página são componentes que podem ser introduzidos na página por um "*layout*" (como vimos anteriormente) ou por outra parcial.

```
1  <div class="navbar megamenu-width">
2      <div class="container">
3          <aside id="main-search">
4              <form action="/{{language}}/search" method="GET">
5                  <a href="/{{language}}" class="close"><i class="icomoon-close"></i></a>
6                  <div class="form-field">
7                      <div class="placeholder">
8                          <label for="query">{{translation.siteSearch}}</label>
9                          <input class="form-control" type="text" maxlength="100" name="query" id="query" />
10                     </div>
11                 </div>
12             </form>
13         </aside>
14         <div class="navbar-inner">
15             <a href="/{{language}}" class="logo">
16                 <img src="/static/images/logo-ita.png" alt="ITA">
17             </a>
18             <ul id="mobile-menu">
19                 <li>
20                     <button class="navbar-toggle" type="button" data-toggle="collapse"
                       ↪  data-target=".navbar-collapse">
21                         <i class="fa fa-reorder"></i>
22                     </button>
23                 </li>
24                 <li><a class="btn-search" href="/{{language}}"><i class="fa fa-search"></i></a></li>
25             </ul>
26             <ul id="main-menu" class="collapse navbar-collapse nav slide">
27                 {{#if config.portal.ita}}
28                     {{> navITA }}
29                 {{/if}}
30
31                 {{> navigator }}
32
33                 {{#if config.portal.contato}}
34                     {{> navContato }}
35                 {{/if}}
36                 <li class="search-nav">
37                     <a href="/{{language}}/search" class="btn-search"><i class="fa fa-search"></i></a>
38                 </li>
39             </ul>
40         </div>
41     </div>
42 </div>
```

Código Fonte 3.10: Parcial contendo a Navegação do Portal Institucional

Na Listagem *3.10* temos a parcial que é inserida pela diretiva da linha 17 em $layout.handlebars$ (ver Listagem *3.9*). Observeque esta parcial aciona diversas outras parciais:

- $navITA$ - linha 28
- $navigator$ - linha 31
- $navContato$ - linha 34

Este recurso do **HandlebarsJS** é utilizando extensivamente pelo portal.

Corpo da página

Conforme observamos na linha 21 do "*layout*" do portal (ver Listagem *3.9*), temos a definição da chave *body* onde se insere o corpo da página com o conteúdo que pretendemos apresentar.

Este conteúdo é apresentado por meio do comando $view.render$ do **CapstoneJS** (ver linha 61 Listagem *3.5*).

O melhor exemplo que podemos apresentar é o de um "*Post*". Ele contém um cabeçalho (ver Figura *3.1*) que apresenta os títulos da publicação (ver linhas 7 até 12 da Listagem *3.11*).

Também, se estiver disponível, um breve resumo da notícia é apresentado (ver linha 11 da Listagem *3.11*) ou o espaço ficará vazio se não.



Figura 3.1: Exemplo de Post

```
1  <div id="page-content" role="main">
2    <div class="container">
3      <div class="container-out">
4        <div class="row">
5          <div class="col-md-9">
6            <div id="content">
7              {{#if data.post.title}}
8              <header class="devider-bottom devider-heavy devider-margin-medium">
9                <h1>{{data.post.title}}</h1>
10               <h3>{{data.post.subtitle}}</h3>
11               <div class="excerpt">{{{data.post.content.brief}}}</div>
12             </header>
```

Código Fonte 3.11: Post do Portal Institucional - Cabeçalho

No corpo da publicação, logo abaixo dos títulos e do resumo, colocamos um vídeo (se houver, ver linhas 16 e 17 da Listagem *3.12*), ou uma imagem (linhas 19 até 21 de Listagem *3.12*).

A partir daí, existe a possibilidade de existir um "*link*" externo com maiores informações (bloco de código que vai da linha 44 até 62) e na linha 64 é acionado uma "parcial" que irá listar arquivos anexos relacionados à esta publicação.

```
13                <article class="post post-single post-type-text">
14                  <div class="post-heading">
15                    <div class="thumbnail">
16                      {{#if data.post.video}}
17                        <iframe width="100%" height="100%" src="{{data.post.video}}" frameborder="0"></iframe>
18                      {{else}}
19                        {{#if data.post.image}}
20                          <div class="slider">
21                            <img src="{{data.post.image.src}}">
22                          </div>
23                        {{/if}}
24                      {{/if}}
25                    </div>
26                  </div>
27                  <div class="post-content">
28                    {{#compare data.post.state "!=" "archived"}}
29                      <div class="content-left">
30                        <div class="article-date">
31                          <span class="type"></span>
32                          <span class="day">{{moment data.post.publishedDate lang="{{language}}"
                            ↪  format="D"}}</span>
33                          <span class="month">{{moment data.post.publishedDate lang="{{language}}"
                            ↪  format="MMM"}}</span>
34                          <span class="year">{{moment data.post.publishedDate lang="{{language}}"
                            ↪  format="YYYY"}}</span>
35                        </div>
36                        <ul class="article-details"></ul>
37                      </div>
38                    {{/compare}}
39                    <div class="content-right">
40                      <div class="text">
41                        {{{data.post.content.extended}}}
42                      </div>
43                    </div>
44                    {{#if data.post.link}}
45                      <div class="call2action call2action-right call2action-colored">
46                        <div class="call2action-heading">
47                          <div class="title">
48                            <h4>{{translation.moreInformation}}</h4>
49                          </div>
50                          <div class="text">
51                            {{#if data.post.content.brief}}
52                              <p>{{{data.post.content.brief}}}</p>
53                            {{else}}
54                              <p>{{{data.post.title}}}</p>
55                            {{/if}}
56                          </div>
57                        </div>
58                        <div class="call2action-content">
59                          <a href="{{data.post.link}}" class="btn btn-big"
                            ↪  target="_blank">{{translation.readMore}}</a>
60                        </div>
61                      </div>
62                    {{/if}}
63                  </div>
64                  {{> fileList }}
65                </article>
```

Código Fonte 3.12: Post do Portal Institucional - Corpo

Finalmente, a publicação termina posicionando a barra lateral de notícias e destaques (se estiver habilitada, ver linha 71 da Listagem *3.13*) e em seguida insere a parcial $subPosts$ que permite que uma publicação tenha diversas outras publicações subordinadas.

```
66                          {{else}}
67                              {{> error }}
68                          {{/if}}
69                      </div>
70                  </div>
71                  {{#eq data.sidebar "true"}}
72                  <div class="col-md-3">
73                      <div class="sidebar" id="sidebar-right">
74                          {{> noticias }}
75                      </div>
76                  </div>
77                  {{/eq}}
78              </div>
79          </div>
80          {{> subPosts }}
81      </div>
82  </div>
```

Código Fonte 3.13: Post do Portal Institucional - Finalização

No Manual do Editor é demonstrado como criar publicações subordinadas. Um exemplo de publicações subordinadas foi utilizado por exemplo pela Pós-graduação.

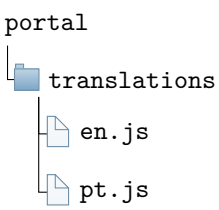
Navegue pelo menu até *Pós-graduação* / *Programas Acadêmicos* / *Programas* / *PG/EAM*. Nesta página, o programa PG/EAM apresenta diversas publicações subordinadas: EAM-1, EAM-2, etc., cada uma representando uma área do programa. Cada uma destas áreas tem um único "*Post*", e todos eles estão subordinados à uma publicação principal que ao ser apresentada encadeia todas as demais. Na Listagem *3.14* mostramos a implementação. Este é um exemplo de customização feita para o portal.

```
{{#if data.child}}
    <div class="container-out container-light">
        <div class="row">
            <div class="col-md-9">
                <div id="content">
                    {{#each data.child}}
                        <div id="content">
                            <header class="devider-bottom devider-heavy devider-margin-medium">
                                <h3>{{this.title}}</h1>
                                <div class="excerpt">{{{this.content.brief}}}</div>
                            </header>
                            <article class="post post-single post-type-text">
                                <div class="post-heading">
                                    <div class="thumbnail">
                                        {{#if this.video}}
                                            <iframe width="100%" height="100%" src="{{this.video}}"
                                            ↪  frameborder="0"></iframe>
                                        {{else if this.image}}
                                            <div class="slider">
                                                <img src="{{this.image.src}}">
                                            </div>
                                        {{/if}}
                                    </div>
                                </div>
                                <div class="post-content">
                                    <div class="content-left">
                                        <div class="article-date">
                                            <span class="type"></span>
                                            <span class="day">{{moment this.publishedDate
                                            ↪  lang="{{../language}}" format="D"}}</span>
                                            <span class="month">{{moment this.publishedDate
                                            ↪  lang="{{../language}}" format="MMM"}}</span>
                                            <span class="year">{{moment this.publishedDate
                                            ↪  lang="{{../language}}" format="YYYY"}}</span>
                                        </div>
                                    </div>
                                    <div class="content-right">
                                        <div class="text">
                                        </div>
                                    </div>
                                    <div class="content-right">
                                        <div class="text">
                                            {{{this.content.extended}}}
                                        </div>
                                    </div>
                                </div>
                            </article>
                        </div>
                    {{/each}}
                </div>
            </div>
        </div>
    </div>
{{/if}}
```

Código Fonte 3.14: Implementação da Parcial de sub-posts

Definição de constantes e a tradução

Um recurso importante, utilizando principalmente pela tradução dos textos do portal que são fixos, é o uso de constantes.



Uma constante é prefixada pelo termo $translation$, e vem normalmente na forma $translation. < constante >$. Na linha 36 da Listagem *3.12* temos por exemplo $translation.moreInformation$, ou seja, será apresentada a versão traduzida da linguagem corrente da constante "$moreInformation$".

A definição da constante está nos arquivos $translations/en.js$ (Listagem *3.15*) e $translations/pt.js$ (Listagem *3.16*).

```
1   module.exports = {
2       root: 'Home',
3       moreInformation: 'More information',
4       readMore: 'Read more...',
5       more: 'more...',
6       language: 'Language',
7       portuguese: 'Portuguese',
8       english: 'English',
9       siteSearch: 'search in this site...',
10      files: 'Files',
```

Código Fonte 3.15: Traduções para Inglês

```
1   module.exports = {
2       root: 'Home',
3       moreInformation: 'Maiores informações',
4       readMore: 'Saiba mais...',
5       more: 'mais...',
6       language: 'Idioma',
7       portuguese: 'Português',
8       english: 'Inglês',
9       siteSearch: 'pesquisar no site...',
10      files: 'Arquivos',
```

Código Fonte 3.16: Traduções para o Português

Os dois arquivos são praticamente idênticos e o que muda é apenas o conteúdo.

Termos comuns estão logo na raíz. Termos específicos de cada página são agrupados dentro de uma chave específica para a página em questão (ver Listagem *3.17*).

```
11      iex: {
12          local: {
13              _title: 'Cooperações',
14              contact: 'Contato',
15          },
16          region: {
17              _title: 'Cooperações',
18              mapHelp: 'Clique na região do mapa na qual deseja encontrar acordos de cooperação',
19              americas: 'Américas',
20              europa: 'Europa & Leste Europeu',
21              asia: 'Ásia & Oceânia',
22              australia: 'Ásia & Oceânia',
23              africa: 'África',
24          },
25      },
```

Código Fonte 3.17: Traduções para o Português

Na Listagem *3.17*, a chave para o termo europa é $translation.iex.region.europa$.

Novas páginas podem ser adicionadas (ou até mesmo termos de menus) incluindo novos valores nos dois arquivos. Também é possível acrescentar traduções (o que vai exigir um pouco mais do que apenas acrescentar um arquivo com as traduções).

Já o conteúdo é traduzido por outro mecanismo, o que pode ser visto no manual do Editor, e é processado pelo componente customizado "*capstone-intl*".

Implementação de métodos

Denominamos métodos requisições especiais que não estão ligadas à uma página diretamente, porém como tudo em **ExpressJS**, toda requisição termina numa resposta que no caso de um portal na internet é sempre uma página.

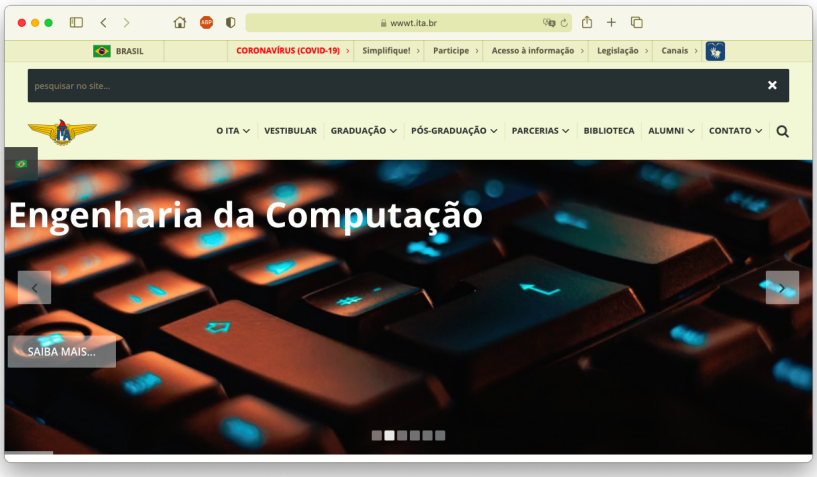


Figura 3.2: Entrada do termo de pesquisa

Um caso comum é o botão de pesquisa (a lupa no canto superior direito que podemos ver na Figura 3.2). Esta opção vai abrir a entrada de valores a serem pesquisados logo acima do menu. Uma vez preenchida, esta entrada irá acionar o método *search* (linha 106 da Listagem 3.2).

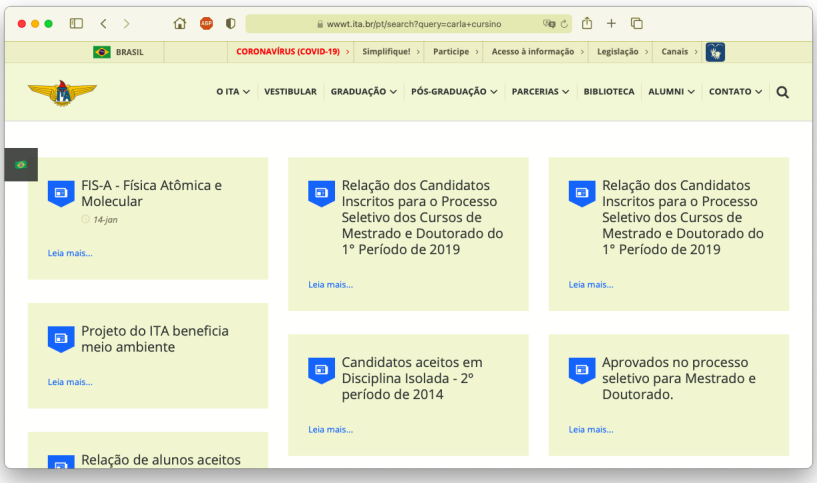


Figura 3.3: Resultado de uma pesquisa

O resultado do método é apresentado como uma página (ver Figura 3.3).

Já o método *download* que é usado por exemplo pelas publicações que apresentam arquivos para o usuário baixar não resulta em uma página, mas sim num processo que resgata um arquivo que o usuário pode baixar em seu computador. Dependendo do browser e configuração do sistema operacional o arquivo pode ser baixado imediatamente ou pode ser solicitado ao usuário indicar a localização da pasta onde o arquivo será gravado.

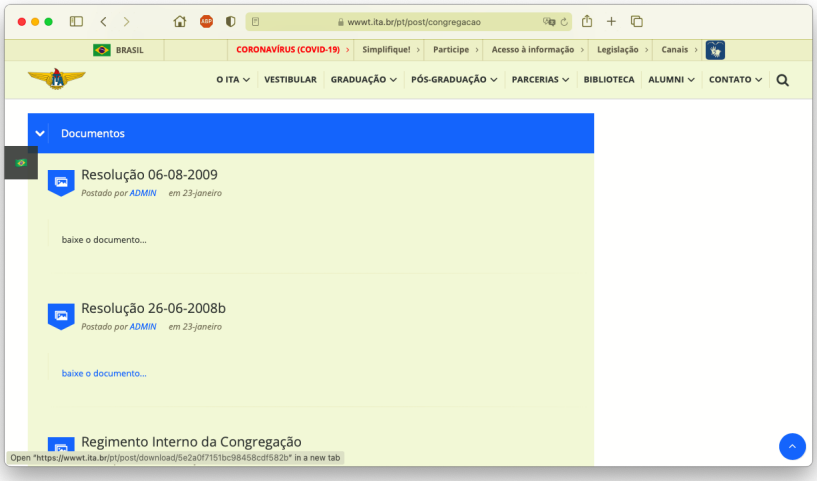


Figura 3.4: Exemplo de publicação com arquivos que podem ser baixados

Definição de modelos de dados

```
config
├── atlas.js
└── default.js
```

Modelos de dados são definições de estruturas dos dados que serão armazenados num banco de dados **MongoDB**. A definição do servidor e banco de dados é feita por meio de arquivos de configuração (ver um exemplo na Listagem 3.18). Já os modelos são definidos em "*scripts*" na pasta *models*.

```
1  module.exports = {
2      cms: {
3          'mongo': 'mongodb+srv://portalAdmin:password@cluster0.xvymj.mongodb.net/dbname?retryWrites=true&w=m
           ↪  ajority',
4          'headless': true,
5          'cookie secret': 'cookie-secret',
6      },
7  }
```

Código Fonte 3.18: Exemplo de definição de servidor e base de dados MongoDB

```
83      },
84      models: './models',
85      routes: './routes',
86  }
```

Código Fonte 3.19: Definição da pasta de modelos de dados

Na Listagem *3.19* temos na linha 84 a definição da pasta onde iremos armazenar nossos modelos. E como de praxe, vamos apresentar como exemplo o modelo de uma publicação.

```
1   var
2       capstone = require('capstonejs'),
3       config = require('config'),
4       translations = require('capstone-intl'),
5       Email = require('keystone-email'),
6       fileManager = require('capstone-file-manager'),
7       Types = capstone.Field.Types,
8       Post = new capstone.List('Post', {
9           map: { name: 'title' },
10          track: true,
11          autokey: { path: 'slug', from: 'title.pt', unique: true },
12          defaultSort: '-publishedDate'
13      })
14
15  new fileManager(Post).init()
16
17  Post.add({
18      title: { type: String, required: true, intl: true },
19      subtitle: { type: String, intl: true },
20      state: { type: Types.Select, options: 'draft, published, archived', default: 'draft', index: true },
21      author: { type: Types.Relationship, ref: 'User', index: true },
22      publishedDate: { type: Types.Date, index: true },
23      approvedDate: { type: Types.Date, index: true },
24      image: { type: Types.File },
25      video: { type: Types.Url },
26      caption: { type: String, intl: true },
27      link: { type: Types.Url },
28      contact: {
29          name: { type: String },
30          email: { type: Types.Email },
31      },
32      content: {
33          brief: { type: Types.Html, wysiwyg: true, height: 150, intl: true },
34          extended: { type: Types.Html, wysiwyg: true, height: 400, intl: true },
35      },
36      categories: { type: Types.Relationship, ref: 'Category', many: true, required: true, initial: true },
37      downloads: { type: Types.Relationship, ref: 'Category', many: false },
38      panel: { type: Types.Select, options: 'acontece, destaque, link, slider, info, grad, posgrad, pg, g,
        ↪  area, visao, valores, coop, noticia', default: 'noticia', index: true, many: true },
39      continent: { type: Types.Select, options: 'americas, europa, asia, africa', index: true },
40      country: { type: String },
41      parent: { type: Types.Relationship, ref: 'Post', index: true },
42  })
43
44  Post.schema.virtual('content.full').get(function() {
45      return this.content.extended || this.content.brief
46  })
47
48  Post.relationship({ ref: 'Slider', refPath: 'post', path: 'sliders' })
49
50  Post.schema.plugin(translations, { languages: config.cms['supported languages'], defaultLanguage:
    ↪  config.cms.language })
51
52  Post.defaultColumns = 'id, slug, title, subtitle, state, publishedDate, categories, downloads, panel'
53
54  Post.register()
```

Código Fonte 3.20: Definição do modelo de postagens

Na Listagem *3.20* podemos ver as definições que serão usadas pelo modelo da linha 1 até 13.

2. Acesso ao gerenciador de conteúdo
3. Acesso às configurações do portal
4. Tradução de conteúdo
5. Tratamento de endereços de e-mail
6. Gerenciamento de arquivos que poderão ser baixados pelo usuário
7. Tipos de dados
8. Definição do repositório de dados no **MongoDB**
9. Nome do atributo a ser utilizado como identificação das publicações
10. Aciona a opção do **Mongoose** de criar atributos que registram data de criação, atualização, usuário que criou e quem atualizou pela última vez
11. Cria uma chave de acesso a partir do título (que irá se chamar $slug$ - este é um nome padrão para "*blogs*")
12. Ordenação padrão a ser usada quando vários registros forem recuperados pelo **Mongoose**

Com base no gerenciador de arquivos definido na linha 6 e do repositório na linha 8, na linha 15 inicializamos o gerenciador de arquivos e da linha 17 até 42 a estrutura de dados que irão compor uma publicação.

18. Título da publicação
19. Sub-título
20. Estado da publicação
    - "*Draft*" - rascunho (padrão) - todo novo registro é criado como "*draft*"
    - "*Published*" - publicada
    - "*Archived*" - arquivada (não mais publicada)
21. Autor da publicação (está relacionado com a tabela de usuários do portal)
22. Data de publicação
23. Data de aprovação da publicação
24. Imagem - é um arquivo que deve ser carregado pelo editor
25. Vídeo - a *URL* que aponta para o vídeo (pode ser no "YouTube" ou "Vimeo")
26. Legenda - um texto da imagem e que aparece em algumas apresentações da publicação
27. Link - uma *URL* para um site externo contendo maiores informações sobre a publicação
28. Contato - dados de contato para o responsável pela publicação
33. Conteúdo resumido - um breve resumo que será apresentado no cabeçalho da publicação e em algumas chamadas para o conteúdo
34. Conteúdo completo - a publicação completa
36. Categorias - relacionamento com a tabela de Categorias, é um elemento importante para a composição de publicações e controle de acesso aos registros publicados

37. Downloads - relacionamento com a tabela de Categorias, indica o grupo de arquivos que será listado para que o usuário possa escolher e baixar em seu computador
38. Panel - painel onde a publicação pode ser apresentada
39. Continente - usado para o agrupamento de publicações para a "IPR"
    - "*Americas*"
    - "*Europa*"
    - "*Asia*"
    - "*Africa*"
40. País - usado para complementar o agrupamento de publicações para a "IPR"
41. Pai - esta é a informação utilizada pelo fragmento $subPosts$ que mostramos anteriormente

Da linha 44 até 46 definimos um atributo virtual (ver documentação do **MongoDB**/**Mongoose** sobre atributos virtuais) que contém todo o texto da publicação (a concatenação de $content.extended$ e $content.brief$).

Em seguida definimos na linha 48 um relacionamento entre a publicação e o documento que irá conter os "*Sliders*" do portal (aquele carrossel de imagens logo no início da página principal).

Na linha 50 registramos o componente de tradução na estrutura de dados, e por fim, na linha 52 definimos as colunas que serão apresentadas nas listas do painel administrativo (ver manual do Editor).

Na linha 54 a estrutura de dados é registrada no **CapstoneJS** que por sua vez utiliza o **Mongoose** para ligá-la ao repositório de documentos no **MongoDB**.

Finalmente, cada atributo possui diversas opções, que devem ser consultadas na documentação do **Mongoose** e **MongoDB**.

Cabeçalhos HTML

A configuração de cabeçalhos *HTML* é necessária para definir regras e proteções para o usuário do Portal.

Estes cabeçalhos são utilizados pelos navegadores para definir o que é permitido ou não pelo site.

Diversas diretivas são implementadas, seguindo-se recomendações de provedores na Internet e fabricantes de navegadores, e a razão para o uso da maioria foge do escopo deste manual. Aqui iremos apenas esclarecer as escolhas feitas para esta versão do Portal.

```
module.exports = {
    headers: {
        hsts: {
            maxAge: 5184000,
            includeSubDomains: false,
        },
        dnsPrefetchControl: {
            allow: false,
        },
        expectCt: {
            enforce: true,
            maxAge: 120,
        },
        frameguard: {
            action: "sameorigin",
        },
        referrerPolicy: {
            policy: "no-referrer",
        },
        permittedCrossDomainPolicies: {
            permittedPolicies: "none",
        },
        contentSecurityPolicy: {
            directives: {
                "default-src": [
                    "'self'",
                ],
                "script-src": [
                    "'self'",
                    "'strict-dynamic'",
                    "'unsafe-inline'",
                    (req, res) => `'nonce-${res.locals.nonce}'`,
                    'https://*.youtube-nocookie.com',
                    'https://maps.google.com',
                    'https://www.google.com',
                    'https://calendar.google.com',
                    'https://barra.brasil.gov.br',
                    'https://vlibras.gov.br',
                ],
                "object-src": ["'none'"],
                "child-src": [
                    "'self'",
                    'https://*.youtube-nocookie.com',
                    'https://maps.google.com',
                    'https://www.google.com',
                    'https://calendar.google.com',
                ],
                "frame-src": [
                    "'self'",
                    'https://*.youtube-nocookie.com',
                    'https://maps.google.com',
                    'https://www.google.com',
                ],
                "style-src": [
                    "'self'",
                    "'unsafe-inline'",
                    'https://fonts.googleapis.com',
                ],
                "font-src": [
                    "'self'",
                    'https://fonts.googleapis.com',
                    'https://barra.brasil.gov.br',
                    'https://fonts.gstatic.com',
                ],
                "img-src": [
                    "'self'",
                    "data:",
                    'https://barra.brasil.gov.br',
                    'https://vlibras.gov.br',
                ],
                "frame-ancestors": ["'self'"],
                "base-uri": ["'self'"],
                upgradeInsecureRequests: [true],
            }
        },
    },
}
```

Código Fonte 3.21: Cabeçalhos **HTML**

Das linhas 3 até 22 temos opções recomendadas para alguns cabeçalhos básicos. Informações detalhadas sobre todos estes cabeçalhos podem ser encontradas no portal Mozilla.

Entre as linhas 23 e 74 temos as definições mais importantes. Aqui são definidos os sites que possuem permissão para inserir "*Scripts*" nas páginas do Portal e possuem portanto o privilégio de execução na página servida aos usuários. Não só "*Scripts*", mas páginas de estilo, fontes e até imagens possuem regras e permissões a serem observadas. Mais uma vez, a documentação do portal Mozilla possui descrição detalhada de como estas informações pode ser modificada com segurança para habilitar ou desabilitar conteúdo de terceiros.

Capítulo 4

Controle de Acessos

4.1 Permissões do Painel Administrativo

4.1.1 Introdução

Usuários



O Painel Administrativo do Portal Institucional possui um cadastro de usuários onde é possível acrescentar, atualizar ou remover usuários com acesso ao próprio painel.

Além do cadastro de usuários no Painel Administrativo, também existe o serviço *LDAP (Lightweight Directory Access Protocol)*[**howes2003understanding**], onde é feita a associação do usuário com as permissões deste no Painel. Para cadastrar um usuário no *LDAP* é possível usar um arquivo *LDIF (LDAP Data Interchange Format)*[**good2000rfc2849**], porém a forma de fazê-lo não importa, desde que algumas informações entre as partes sejam comuns sejam observadas tais como a identificação do usuário por meio do seu "*email*" e de pertencer à determinados grupos de usuários.

Outra opção é a associação do usuário feita por meio de configurações no próprio cadastro deste

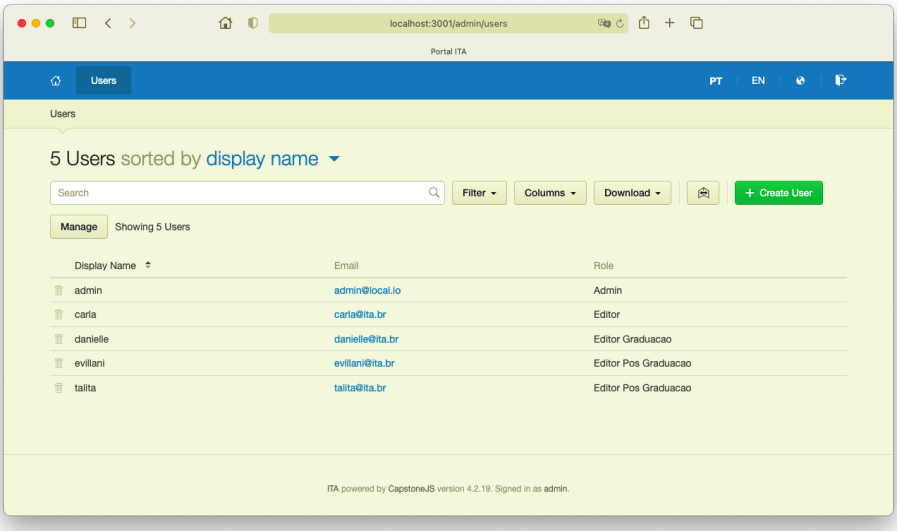


Figura 4.1: Lista de usuários cadastrados

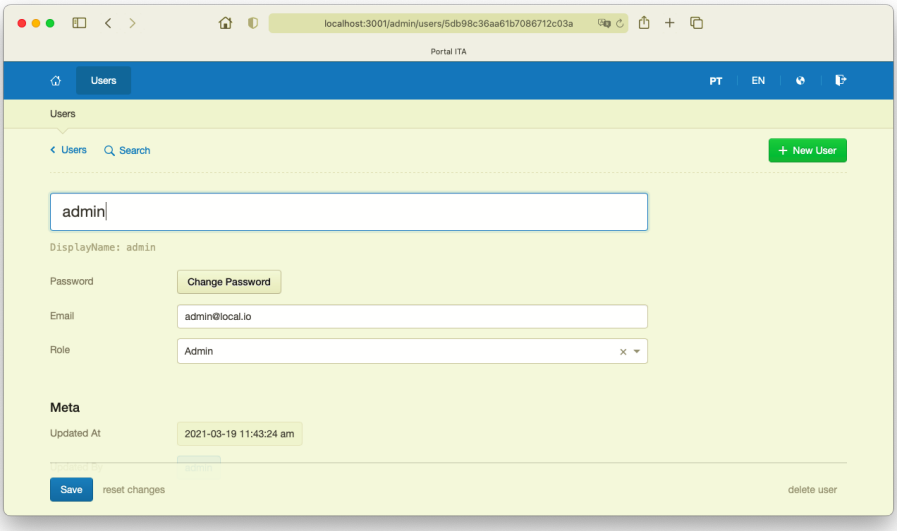


Figura 4.2: Tela do cadastro de usuários

no Painel Administrativo.

```
dn: uid=admin@local.io,dc=portal,dc=ita,dc=br
uid: admin@local.io
userPassword: p-a-s-s-w-o-r-d
objectClass: account
objectClass: simpleSecurityObject
objectClass: top
```

Código Fonte 4.1: Exemplo de arquivo *LDIF*

Primeiramente, nas linhas 1 e 2 na Listagem *4.1*, os atributos "dn" e "uid" devem conter o mesmo que foi informado na tela da Figura *4.2*, em "*Email*". Sem isso não é possível ao Controle de Acesso do Painel Administrativo localizar o usuário.

E também observar que o atributo "$userPassword$" existe somente por conformidade com o padrão do arquivo *LDIF*, e de modo algum esta senha será utilizada para autenticar o usuário. A autenticação de usuários é feita atualmente pelo próprio Painel Administrativo.

Recursos



Os recursos do Portal Institucional que são administrados pelo Controle de Acesso são informações armazenadas em bancos de dados. Estas informações contém a estrutura e os dados que aparecem nas páginas do Portal, e são descritas seguindo especificações do banco de dados MongoDB[**chodorow2013mongodb**].

```
1  var
2      capstone = require('capstonejs'),
3      config = require('config'),
4      translations = require('capstone-intl'),
5      Types = capstone.Field.Types,
6      Category = new capstone.List('Category', {
7          map: { name: 'name' },
8          track: true,
9          autokey: { path: 'slug', from: 'name.pt', unique: true }
10     })
11
12 Category.add({
13     name: { type: String, required: true, intl: true },
14     description: { type: Types.Html, wysiwyg: true, height: 150, intl: true },
15 })
16
17 Category.relationship({ ref: 'Post', refPath: 'categories', path: 'posts' })
18
19 Category.relationship({ ref: 'Menu', refPath: 'category', path: 'menus' })
20
21 Category.relationship({ ref: 'Archive', refPath: 'categories', path: 'archives' })
22
23 Category.schema.plugin(translations, { languages: config.cms['supported languages'],
   ↪  defaultLanguage: config.cms.language })
24
25 Category.defaultColumns = 'id, name, description'
26
27 Category.register()
```

Código Fonte 4.2: Descrição do documento de "Categories" do Portal Institucional

Na definição de recursos existem algumas características importantes. Entre as linhas 1 e 10 temos a inicialização de definições utilizadas pelo documento. Em particular, da linha 6 até 10 temos uma definição geral do documento. Em seguida, da linha 12 até 15 a estrutura do documento, ou seja, seus atributos. Para maiores informações sugerimos a leitura do livro de **chodorow2013mongodb** (**chodorow2013mongodb**) que referenciamos anteriormente.

Podemos destacar que, na linha 4 foi introduzido um componente customizado denominado "*capstone−intl*", que é configurado na linha 23. Esta customização foi necessária pois em condições normais o banco de dados MongoDB não suporta múltiplos idiomas.

Também queremos chamar atenção para o trecho que vai da linha 17 até a linha 21. Neste ponto são descritos relacionamentos que outros documentos fazem com "Categorias". Isto será mais tarde referenciado na configuração de controles de acesso.

Além das "Categorias", existem diversos outros recursos, todos sob a tutela do Controle de Acesso, e sobre os quais permissões devem ser dadas para que o usuário possa incluir ou alterar informações do Portal Institucional.

4.1.2 Perfis

Perfis agrupam diferentes tipos de usuários do Painel Administrativo. Observe que o Portal Institucional não possui usuários autenticados, sendo um portal de acesso público na internet, não existem e nem são categorizados perfis de usuários do portal. Porém o Painel Administrativo é de acesso restrito a funcionários do *ITA (Instituto Tecnológico de Aeronáutica)*, e para acessá-lo é necessário que a pessoa fazendo-o esteja previamente cadastrada.



Este cadastro é feito simultaneamente no próprio Painel Administrativo, por meio de um usuário primordial denominado "*admin*". Este usuário especial é criado quando o Portal é inicializado pela primeira vez. A senha para este usuário é definida no arquivo "$0.0.1 - first - user.js$" de configuração, na pasta "$updates$" (ver Listagem 4.3).

```
1  exports.create = {
2    User: [
3      {
4        displayName: 'admin',
5        email: 'admin@local.io',
6        password: '-s-e-c-r-e-d-o-',
7        role: 'admin',
8      },
9    ],
10 }
```

Código Fonte 4.3: Arquivo de configuração $0.0.1 - first - user.js$

Veja na linha 6 a senha informada. Esta senha deverá ser informada na tela de acesso ao Painel Administrativo (ver Figura 4.3). Recomenda-se que a senha seja modificada logo após o primeiro acesso.

Integração de perfis via LDAP

Em seguida, o usuário "$admin$" deve criar e manter os demais usuários que terão acesso ao Portal Administrativo. Em condições normais, nenhum usuário adicional deverá ter permissão para criar ou modificar os dados dos demais usuários (veja Figura 4.1).

A criação de perfis se dará também em duas etapas. Primeiramente é necessário definir os perfis desejados, e em seguida criar o grupo para cada perfil no servidor *LDAP*. A definição de perfis se dá pela definição de regras de acesso no arquivo "$rules.js$" na pasta "$config$" contendo regras de acesso, o que veremos a seguir.

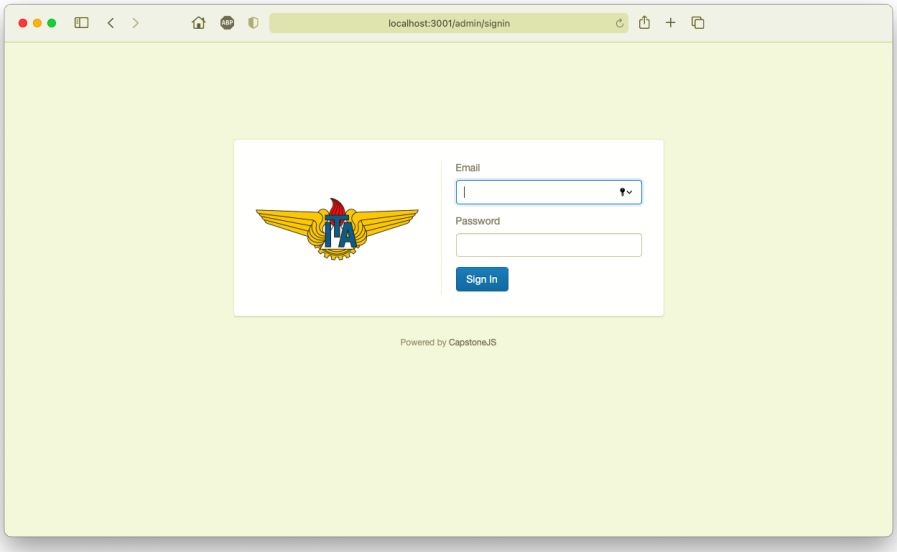


Figura 4.3: Tela de acesso ao Painel Administrativo

```
1  dn: cn=editor,dc=portal,dc=ita,dc=br
2  cn: editor
3  member: uid=maria@local.io
4  objectClass: groupOfNames
5  objectClass: top
6
7  dn: cn=editor-graduacao,dc=portal,dc=ita,dc=br
8  cn: editor-graduacao
9  member: uid=sandra@local.io
10 member: uid=rita@local.io
11 member: uid=ricardo@local.io
12 objectClass: groupOfNames
13 objectClass: top
14
15 dn: cn=editor-pos-graduacao,dc=portal,dc=ita,dc=br
16 cn: editor-pos-graduacao
17 member: uid=teresa@local.io
18 member: uid=joao@local.io
19 objectClass: groupOfNames
20 objectClass: top
```

Código Fonte 4.4: Exemplo de definição de perfis para o LDAP

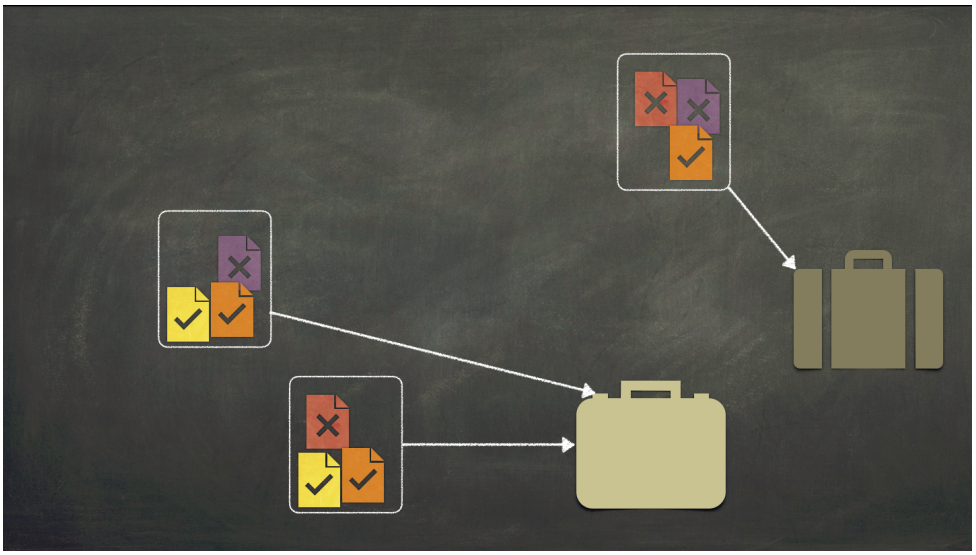
Na Listagem 4.4 informamos ao serviço *LDAP* os perfis que serão reconhecidos pelo controle de acesso. Observe também que os usuários para cada perfil já foram também informados (por exemplo, linhas 3, 9 até 11 e 17 até 18).

Integração de perfis via DB

Na Figura 4.2 podemos ver o atributo "*Role*", onde são listados os papéis encontrados na configuração de **Regras de Acesso** (que veremos logo a seguir). Caso não haja um servidor **LDAP** disponível (ver (ver Guias do Desenvolvedor e do Operador), então este será o papel utilizado pelo para definir as regras de acesso do usuário.

Este atributo apresenta uma lista e o papel desejado pode ser escolhido. Diferentemente da definição de papéis via serviço **LDAP**, apenas um papel por usuário pode ser atribuído por intermédio do banco de dados.

4.1.3 Regras de Acesso



Como dissemos anteriormente, a definição de perfis se dá pela definição de regras de acesso no arquivo "$rules.js$" na pasta "$config$" contendo regras de acesso.

Na Listagem 4.4 podemos ver a definição do perfil "$editor$" que será atribuído ao usuário "$maria$" - a nossa editora geral e atribuida ao perfil na linha 3. Esta definição reside no servidor *LDAP*, onde temos as atribuições de usuários e os perfis correspondentes.

Porém, a definição de como o perfil deve ser atendido na aplicação é definida por meio de regras de acesso no arquivo "$rules.js$" na pasta "$config$". Na Listagem 4.5 mostramos uma definição de regras para o perfil "$editor$".

```
79              'cn=editor,dc=portal,dc=ita,dc=br': {
80                  'archives': {
81                      'update:any': [ '*' ],
82                      'delete:any': [ '*' ],
83                  },
84                  'archives.createdBy': {
85                      'reference:any': [ { key: 'createdBy', value: '*' }, ],
86                  },
87                  'archives.updatedBy': {
88                      'reference:any': [ { key: 'updatedBy', value: '*' }, ],
89                  },
90                  'categories': {
91                      'update:any': [ '*' ],
92                      'delete:any': [ '*' ],
93                  },
94                  'categories.createdBy': {
95                      'reference:any': [ { key: 'createdBy', value: '*' }, ],
96                  },
97                  'categories.updatedBy': {
98                      'reference:any': [ { key: 'updatedBy', value: '*' }, ],
99                  },
100                 'menus': {
101                     'update:any': [ '*' ],
102                     'delete:any': [ '*' ],
103                 },
104                 'menus.createdBy': {
105                     'reference:any': [ { key: 'createdBy', value: '*' }, ],
106                 },
107                 'menus.updatedBy': {
108                     'reference:any': [ { key: 'updatedBy', value: '*' }, ],
109                 },
110                 'menus.post': {
111                     'reference:any': [ { key: 'post', value: '*' }, ],
112                 },
113                 'menus.category': {
114                     'reference:any': [ { key: 'category', value: '*' }, ],
115                 },
116                 'posts': {
117                     'update:any': [ '*' ],
118                     'delete:any': [ '*' ],
119                 },
120                 'posts.author': {
121                     'reference:any': [ { key: 'author', value: '*' }, ],
122                 },
123                 'posts.categories': {
124                     'reference:any': [ { key: 'categories', value: '*' }, ],
125                 },
126                 'posts.downloads': {
127                     'reference:any': [ { key: 'downloads', value: '*' }, ],
128                 },
129                 'posts.parent': {
130                     'reference:any': [ { key: 'parent', value: '*' }, ],
131                 },
132                 'posts.createdBy': {
133                     'reference:any': [ { key: 'createdBy', value: '*' }, ],
134                 },
135                 'posts.updatedBy': {
136                     'reference:any': [ { key: 'updatedBy', value: '*' }, ],
137                 },
138             },
```

Código Fonte 4.5: Exemplo de regras de controle de acesso

Na linha 79 temos a definição do perfil, seguindo a estrutura de identificadores do serviço *LDAP*. Em seguida temos o nome do recurso (exemplo na linha 80 até 83, "*archives*") e em seguida as permissões

do perfil. As permissões possíveis são:

- "$update : any$" - Atualização;
- "$delete : any$" - Exclusão;

Além destas permissões, o controle de acesso poderia controlar permissões para "*Leitura*" ("$read : any$") e "*Inclusão*" ("$create : any$"), porém este tipo de acesso não é considerado pelo Painel Administrativo.

Além disto, o termo "any" chama a atenção. Existe ainda a possibilidade de definir se o acesso é de um recurso de todos usuários ("*any*") ou próprio ("*own*"), porém, mais uma vez, o Painel Administrativo não considera a propriedade do recurso.

Finalmente, também podemos observar o valor "$['*']$" informado para o acesso. Seria possível informar uma lista de atributos do recurso para o qual o usuário teria acesso, e novamente, o Painel Administrativo não considera a segregação de atributos do recurso.

Em seguida, das linhas 84 até 89 temos definições de recursos com o nome em duas partes. Na primeira parte temos o nome do recurso (em nosso caso "$archives$") e na segunda parte um relacionamento deste recurso com outro recurso do Portal Administrativo. Temos por exemplo "$archives.createdBy$" e o acesso "$reference : any$". Antes de mais nada, veja na Listagem 4.6 a definição de "$archives$".

```
1  var
2      capstone = require('capstonejs'),
3      config = require('config'),
4      translations = require('capstone-intl'),
5      fileManager = require('capstone-file-manager'),
6      Types = capstone.Field.Types,
7      Archive = new capstone.List('Archive', {
8          map: { name: 'fileName' },
9          track: true,
10     })
11
12 new fileManager(Archive).init()
13
14 Archive.add({
15     fileName: { type: String, required: true },
16     caption: { type: String },
17     description: { type: Types.Html, wysiwyg: true, height: 150, intl: true },
18     archive: { type: Types.File },
19     categories: { type: Types.Relationship, ref: 'Category', many: true },
20     enabled: { type: Boolean, default: true },
21 })
22
23 Archive.schema.plugin(translations, { languages: config.cms['supported languages'],
   ↪ defaultLanguage: config.cms.language })
24
25 Archive.defaultColumns = 'id, fileName, caption, categories'
26
27 Archive.register()
```

Código Fonte 4.6: Modelo do recurso "$Archives$"

Na linha 9 temos a definição "$track : true$", que indica ao Painel Administrativo que desejamos ter registrados o nome do usuário que criou o registro ("$createdBy$"), do último usuário que o atualizou ("$updatedBy$") e quando isto ocorreu ("$createdAt$" e "$updateAt$" respectivamente). Estes atributos são diretamente ligados ao cadastro de usuários do Portal Administrativo.

Já um exemplo de ligação entre um documento e outro pode ser observado neste mesmo modelo de "$archives$" na linha 19, onde existe a definição de uma relação entre estes "*archives*" e o documento de "*Categories*" (veja na linha 21 na Listagem 4.2 a mesma definição, porém dentro do documento referenciado).

Este tipo de ligação existe portanto entre "*Archives*" e os usuários do Portal Administrativo, e assim sendo é necessária a permissão para que quem for atualizar "*Archives*" possa também atualizar esta relação.

4.1.4 Segregação de Acesso

Um dos principais objetivos de se ter um controle de acessos, é definir quem pode publicar no Portal Administrativo, e onde poderá publicá-las.

Para isto utilizaremos o recurso de definir regras de acesso e restrições no relacionamento do acesso.

No exemplo dado na Listagem 4.5, o acesso "$reference : any$" recebe os valores "$key :' createdBy', value :' *'$", que basicamente indica que o usuário pode referenciar qualquer valor que for atribuido em "**createdBy**" no documento "$archives$".

Porém para restringir a publicação no Portal Institucional, utilizaremos outro recurso que é informar um valor exato no par de atributos "key" e "$value$".

Isto irá funcionar pois daremos apenas aos usuário com perfil "*editor*" a permissão de atualizar "*Menus*", e em seguida definiremos exatamente o que cada menu irá apresentar por meio do atributo "*Categories*", que define um grupo para classificar as informações publicadas no Portal Institucional.

Finalmente, será definido um perfil de usuário com permissão de criação de relações entre as publicações e determinadas categorias, como na Listagem 4.7 que define uma regra que permite apenas publicar na categoria denominada "$graduacao$", em contraste com a Listagem 4.8, que permite publicar com qualquer categoria.

```
155                 'posts.categories': {
156                     'reference:any': [ { key: 'categories', value: 'graduacao' }, ],
157                 },
```

Código Fonte 4.7: Exemplo de regras de controle de acesso

```
123                 'posts.categories': {
124                     'reference:any': [ { key: 'categories', value: '*' }, ],
125                 },
```

Código Fonte 4.8: Exemplo de regras de controle de acesso

Além disto, o atributo "$category$" é obrigatório para as publicações (ver linha 36 da Listagem 4.9 onde informamos "$required : true, initial : true$", que indica que o atributo é obrigatório e deve ser informado antes de ser possível criar um novo registro).

```
var
    capstone = require('capstonejs'),
    config = require('config'),
    translations = require('capstone-intl'),
    Email = require('keystone-email'),
    fileManager = require('capstone-file-manager'),
    Types = capstone.Field.Types,
    Post = new capstone.List('Post', {
        map: { name: 'title' },
        track: true,
        autokey: { path: 'slug', from: 'title.pt', unique: true },
        defaultSort: '-publishedDate'
    })

new fileManager(Post).init()

Post.add({
    title: { type: String, required: true, intl: true },
    subtitle: { type: String, intl: true },
    state: { type: Types.Select, options: 'draft, published, archived', default: 'draft',
    ↪  index: true },
    author: { type: Types.Relationship, ref: 'User', index: true },
    publishedDate: { type: Types.Date, index: true },
    approvedDate: { type: Types.Date, index: true },
    image: { type: Types.File },
    video: { type: Types.Url },
    caption: { type: String, intl: true },
    link: { type: Types.Url },
    contact: {
        name: { type: String },
        email: { type: Types.Email },
    },
    content: {
        brief: { type: Types.Html, wysiwyg: true, height: 150, intl: true },
        extended: { type: Types.Html, wysiwyg: true, height: 400, intl: true },
    },
    categories: { type: Types.Relationship, ref: 'Category', many: true, required: true,
    ↪  initial: true },
    downloads: { type: Types.Relationship, ref: 'Category', many: false },
    panel: { type: Types.Select, options: 'acontece, destaque, link, slider, info, grad,
    ↪  posgrad, pg, g, area, visao, valores, coop, noticia', default: 'noticia', index:
    ↪  true, many: true },
    continent: { type: Types.Select, options: 'americas, europa, asia, africa', index: true },
    country: { type: String },
    parent: { type: Types.Relationship, ref: 'Post', index: true },
})

Post.schema.virtual('content.full').get(function() {
    return this.content.extended || this.content.brief
})

Post.relationship({ ref: 'Slider', refPath: 'post', path: 'sliders' })

Post.schema.plugin(translations, { languages: config.cms['supported languages'],
↪  defaultLanguage: config.cms.language })

Post.defaultColumns = 'id, slug, title, subtitle, state, publishedDate, categories,
↪  downloads, panel'

Post.register()
```

Código Fonte 4.9: Modelo de Publicação

Herdando regras

É possível facilitar a criação de novos grupos de regras agrupando as regras comuns num único grupo e utilizando a cláusula “”$extend” : [’< grupo − base >’]”, como no exemplo da Listagem 4.10.

```
1  /**
2   *
3   * Regras de controle de acesso.
4   *
5   */
6  module.exports = {
7      cms: {
8          'access rules': {
9              'editor-base': {
10                 'archives': {
11                     'update:any': [ '*' ],
12                 },
13                 'archives.createdBy': {
14                     'reference:any': [ { key: 'createdBy', value: '*' }, ],
15                 },
16                 'archives.updatedBy': {
17                     'reference:any': [ { key: 'updatedBy', value: '*' }, ],
18                 },
19                 'posts': {
20                     'update:any': [ '*' ],
21                 },
22                 'posts.author': {
23                     'reference:any': [ { key: 'author', value: '*' }, ],
24                 },
25                 'posts.downloads': {
26                     'reference:any': [ { key: 'downloads', value: '*' }, ],
27                 },
28                 'posts.parent': {
29                     'reference:any': [ { key: 'parent', value: '*' }, ],
30                 },
31                 'posts.createdBy': {
32                     'reference:any': [ { key: 'createdBy', value: '*' }, ],
33                 },
34                 'posts.updatedBy': {
35                     'reference:any': [ { key: 'updatedBy', value: '*' }, ],
36                 },
37             },
38             'editor-ipr': {
39                 "$extend": [ 'editor-base' ],
40                 'archives.categories': {
41                     'reference:any': [ { key: 'categories', value: 'parcerias' }, ],
42                 },
43                 'posts.categories': {
44                     'reference:any': [ { key: 'categories', value: 'parcerias' }, ],
45                 },
46             },
47         },
48     }
49 }
```

Código Fonte 4.10: Exemplo de regras de controle de acesso comuns

Na linha 39 podemos ver que $editor - ipr$ “extende” as regras de “$editor - base$”.

4.1.5 Regras de Validação

Uma extenção ao Controle de Acesso são as regras de validação.

Isto permite implementar papéis que possuem permissões específicas para determinados valores em atributos dos dados.

Para implementar um perfil que por exemplo possui permissão para inserir publicações com determinados valores, e outro perfil com permissões complementares, podemos fazer o seguinte:

```
1  /**
2   * Regras de controle de acesso.
3   */
4  module.exports = {
5      cms: {
6          'access rules': {
7              'editor-graduacao': {
8                  "$extend": [ 'editor-base' ],
9                  'posts.categories': {
10                     'reference:any': [ { key: 'categories', value: 'graduacao' }, ],
11                 },
12             },
13             'revisor-graduacao': {
14                 "$extend": [ 'editor-graduacao' ],
15             },
16     }
17 }
```

Código Fonte 4.11: Perfis com regras de validação

Entre as linhas 7 e 12 definimos um papel **editor-graduacao** e entre as linhas 13 e 15 o extendemos com exatamente as mesmas atribuições para o papel **revisor-graduacao**.

Na listagem abaixo complementamos os dois papéis definidos acima com regras de validação específicas para cada um dos papéis:

```
1  /**
2   * Regras de controle de acesso.
3   */
4   const
5      ALLOW = true,
6      DENY = false
7
8  module.exports = {
9      cms: {
10         'validation rules': {
11             'editor-graduacao': {
12                 'posts': [
13                     { 'isIn': { valid: ALLOW, 'state': ['draft', 'archived'] } },
14                     { 'isIn': { valid: DENY, 'state': ['published'] } },
15                 ],
16             },
17             'revisor-graduacao': {
18                 'posts': [
19                     { 'isIn': { valid: ALLOW, 'state': ['draft', 'published', 'archived'] } },
20                 ],
21             },
22     }
23 }
```

Código Fonte 4.12: Perfis com regras de validação

Na linha 13, o **editor-graduacao** implementa uma regra que diz:

> *Permitir (ALLOW) Publicações (posts) que possuem os textos (isIn) "draft" e "archived" no atributo "state"*

Já a linha 14 o perfil implementa a regra:

> *Negar (DENY) Publicações (posts) que possuem o texto (isIn) "published" no atributo "state"*

Em linhas gerais estamos **permitindo** ao perfil **editor-graduacao** possa gravar **publicações** com **estado rascunho** ou **arquivado** e **negando** no estado **publicado**.

Já o perfil **revisor-graduacao** implementa a regra:

> *Permitir (ALLOW) Publicações (posts) que possuem os textos (isIn) "draft", "published" e "archived" no atributo "state"*

Neste caso estamos **permitindo** ao perfil **revisor-graduacao** gravar **publicações** em **qualquer** um dos **estado** possíveis.